

A Pattern-Driven Gamification Framework for Interaction and Visualization in Programmable Reality Environments

Grigoriou Eleni



Thesis submitted in partial fulfillment of the requirements for the
Masters' of Science degree in Computer Science and Engineering

University of Crete
School of Sciences and Engineering
Computer Science Department
Voutes University Campus, 700 13 Heraklion, Crete, Greece

Thesis Advisor: Professor *George Papagiannakis*

This work has been performed at the University of Crete, School of Sciences and Engineering, Computer Science Department.

The work has been supported by the Foundation for Research and Technology - Hellas (FORTH), Institute of Computer Science (ICS).

UNIVERSITY OF CRETE
COMPUTER SCIENCE DEPARTMENT

**A Pattern-Driven Gamification Framework for Interaction and
Visualization in Programmable Reality Environments**

Thesis submitted by
Grigoriou Eleni
in partial fulfillment of the requirements for the
Masters' of Science degree in Computer Science

THESIS APPROVAL

Author: _____
Grigoriou Eleni

Committee approvals: _____
George Papagiannakis
Professor, Thesis Supervisor

Panos Trahanias
Professor, Committee Member

Asterios Leonidis
Assistant Professor, Committee Member

Departmental approval: _____
Grigorios Tsagakatakis
Assistant Professor, Director of Graduate Studies

Heraklion, October 2025

A Pattern-Driven Gamification Framework for Interaction and Visualization in Programmable Reality Environments

Abstract

Programmable reality envisions digital environments where guidance, feedback, and motivation are not scripted manually but generated dynamically from declarative design patterns. This thesis contributes to that vision by introducing a pattern-driven gamification framework for programmable reality, developed in the context of Extended Reality (XR) training.

The framework unifies two complementary domains: spatial visualization patterns, which define how feedback and instructional cues appear in 3D space (Ghost, Label, Panel, Glyph, Virtual Mirror, Trajectory) and procedural interaction patterns, which capture learner actions (Insert, Remove, Use). By embedding these patterns directly into the authoring process, instructional feedback and gamified guidance become intrinsic parts of scenario design. A no-code authoring methodology enables creators to compose and adapt such patterns visually, ensuring consistency, reusability, and pedagogical soundness across programmable-reality training experiences.

Beyond consolidating existing design vocabularies, the work introduces several new interaction patterns, Trajectory, Question, Activate, Choice Action and complementary visualization patterns such as Motion and Progress Indicator.

The main contribution of this thesis is a framework and methodology that unify spatial visualization and procedural interaction patterns into a coherent model for authoring programmable-reality training systems. Together, these components support the broader concept of programmable reality, immersive simulations in which feedback, guidance, and motivation are systematically encoded as modular design elements.

Ένα Πλαίσιο Παιχνιδοποίησης Καθοδηγούμενο από Πρότυπα Σχεδίασης Λογισμικού για την αλληλεπίδραση και οπτικοποίηση σε περιβάλλοντα Προγραμματιζόμενης Πραγματικότητας

Περίληψη

Η προγραμματιζόμενη πραγματικότητα (programmable reality) περιγράφει ψηφιακά περιβάλλοντα στα οποία η καθοδήγηση, η ανατροφοδότηση και οι στόχοι του χρήστη δεν σχεδιάζονται χειροκίνητα, αλλά παράγονται δυναμικά μέσα από πρότυπα σχεδίασης λογισμικού.

Η παρούσα εργασία συμβάλλει σε αυτήν την νέα ερευνητική κατεύθυνση, παρουσιάζοντας ένα πλαίσιο παιχνιδοποίησης καθοδηγούμενο από συγκεκριμένα υπάρχοντα και νέα πρότυπα σχεδίασης λογισμικού για την προγραμματιζόμενη πραγματικότητα, το οποίο αναπτύχθηκε στο πλαίσιο εκπαιδευτικών εμπειριών σε περιβάλλοντα Εκτεταμένης Πραγματικότητας (Extended Reality – XR).

Το προτεινόμενο πλαίσιο ενοποιεί δύο συμπληρωματικούς τομείς:

(α) τα πρότυπα χωρικής οπτικοποίησης, που καθορίζουν τον τρόπο με τον οποίο η ανατροφοδότηση και τα διδακτικά στοιχεία προβάλλονται στον τρισδιάστατο χώρο (π.χ. Ghost, Label, Panel, Glyph, Virtual Mirror, Trajectory), και

(β) τα πρότυπα διαδικασιακής αλληλεπίδρασης, που καταγράφουν τις ενέργειες του εκπαιδευόμενου (Insert, Remove, Use).

Με την ενσωμάτωση αυτών των προτύπων απευθείας στη διαδικασία δημιουργίας, η διδακτική ανατροφοδότηση και η καθοδήγηση του χρήστη καθίστανται εγγενή στοιχεία του σχεδιασμού των σεναρίων. Μια μεθοδολογία δημιουργίας χωρίς κώδικα (no-code authoring) επιτρέπει στους δημιουργούς να συνθέτουν και να προσαρμόζουν οπτικά τα πρότυπα αυτά, εξασφαλίζοντας συνέπεια, επαναχρησιμοποίηση και παιδαγωγική ορθότητα στις εμπειρίες εκπαίδευσης προγραμματιζόμενης πραγματικότητας.

Πέρα από την ενοποίηση υπάρχοντων προτύπων οπτικοποίησης, η εργασία εισάγει νέα πρότυπα αλληλεπίδρασης (Trajectory, Question, Activate, Choice Action), καθώς και συμπληρωματικά πρότυπα οπτικοποίησης, όπως Motion και Progress Indicator.

Η κύρια συνεισφορά της εργασίας είναι η ανάπτυξη ενός ενιαίου πλαισίου και μιας μεθοδολογίας που ενοποιούν τα πρότυπα χωρικής οπτικοποίησης και διαδικασιακής αλληλεπίδρασης σε ένα ενιαίο μοντέλο σχεδίασης προτύπων λογισμικού για τη δημιουργία εκπαιδευτικών συστημάτων προγραμματιζόμενης πραγματικότητας. Συνολικά, τα στοιχεία αυτά υποστηρίζουν την ευρύτερη έννοια της προγραμματιζόμενης πραγματικότητας: εμβυθιστικές προσομοιώσεις στις οποίες η ανατροφοδότηση, η καθοδήγηση και οι στόχοι του χρήστη κωδικοποιούνται συστηματικά ως αρθρωτά στοιχεία ενιαίας σχεδίασης.

Publications Arisen from This Work

The following publications have arisen in whole or in part from the research presented in this thesis. They include both foundational works that established the methodological basis for the present framework and broader collaborative efforts that extended its application within the domain of Computational Medical XR.

- Zikas, P., Protopsaltis, A., Lydatakis, N., Kentros, M., Geronikolakis, S., Kateros, S., Kamarianakis, M., Evangelou, G., Filippidis, A., **Grigoriou, E.**, Angelis, D., Tamiolakis, M., Dodis, M., Kokiadis, G., Petropoulos, J., Pateraki, M., & Papagiannakis, G. (2023). *MAGES 4.0: Accelerating the World's Transition to VR Training and Democratizing the Authoring of the Medical Metaverse*. *IEEE Computer Graphics and Applications*, 43(2), 43–56. <https://doi.org/10.1109/MCG.2023.3242686> [This publication established the foundational scenegraph-based authoring framework for XR training and introduced the first set of interaction patterns (Insert, Use, Remove). The present thesis extends this foundation by introducing new procedural patterns (Activate, Trajectory, Question, Choice) and formalizing gamification as an integrated, pattern-driven subsystem within programmable reality.]
- Papagiannakis, G., Greenleaf, W., Cole, M., Zhang, M., Datta, R., Delahaye, M., **Grigoriou, E.**, Kamarianakis, M., Protopsaltis, A., Bijlenga, P., Magnenat-Thalmann, N., Tsiridis, E., Kenanidis, E., Vamvakidis, K., Koutelidakis, I., & Kannape, O. A. (2025). *A Computational Medical XR Discipline*. In N. Magnenat-Thalmann, J. Kim, B. Sheng, Z. Deng, D. Thalmann, & P. Li (Eds.), *Advances in Computer Graphics. CGI 2024* (Lecture Notes in Computer Science, Vol. 15339, pp. 43–56). Springer, Cham. https://doi.org/10.1007/978-3-031-82021-2_4 [This publication articulates the broader vision of a unified Computational Medical XR discipline, emphasizing the need for low-code authoring and gamified training frameworks. The present thesis directly contributes to this research direction by implementing a concrete authoring subsystem that operationalizes these principles through pattern-driven interaction and gamification design.]

Acknowledgements

I would first like to express my deepest gratitude to my supervisor, Professor George Papagiannakis, for his tireless support, guidance, and inspiration throughout this project. His trust in me—since my undergraduate years—opened the door to invaluable experiences and opportunities to work alongside an extraordinary team. His vision, encouragement, and belief in my abilities have shaped both this thesis and my personal and professional growth.

I am also sincerely thankful to my colleagues Michael Dodis, Steve Kateros, Nikos Lydatakis, and Amalia Kargopoulou. Michael’s programming expertise and his contribution to the Gamification Subsystem were instrumental to this work. His deep technical insight, problem-solving ability, and humor provided much-needed clarity and motivation during challenging periods. Steve’s openness to brainstorming and design discussions helped refine many of the ideas that guided this project. Nikos and Amalia have been a constant source of motivation, raising the standards of our work and inspiring me to keep pushing forward.

I would also like to thank Eve for her friendship, encouragement, and unwavering support—especially in the moments that mattered most.

Finally, my deepest thanks go to my family. Konstantinos, Katholiki, and Grigoris, for their unconditional love, patience, and faith in me. None of this would have been possible without their constant support and understanding.

Contents

1	Introduction	1
2	Background and Related Work	5
2.1	Programmable Reality	5
2.2	Extended Reality and Serious Games	7
2.3	Gamification in Serious Games	8
2.4	Gamification in XR Training	9
2.5	From Manual Scripting to Algorithmic Authoring	10
2.6	Frameworks and Theoretical Basis	11
2.6.1	Gamification Theory	12
2.6.2	Situated Visualization Patterns	12
2.6.3	Learning Science Foundations	12
2.6.4	The Pillars of Programmable Reality	13
2.7	Design Patterns in XR and HCI	13
2.8	XR Authoring Tools and Visual Scripting	14
2.9	Scenegraph Architecture and Interaction Patterns	15
2.10	Motivation	16
2.11	Prior Research and Related Work	17
2.12	Gamification in Learning and Training	18
2.13	Instructional and Cognitive Design in XR	18
2.14	Design Patterns and Guidance in XR	19
2.15	Scenegraph Authoring and System Extensions	20
2.16	Research Gap and Aim	20
3	Methodology	23
3.1	Overview	23
3.2	Conceptual Foundations	24
3.3	Methodological Approach	26
3.4	Development Process	27
3.5	System Architecture and Implementation	31
3.5.1	Relation to the Underlying Scenegraph Framework	31
3.5.2	Gamification Subsystem Data Model	31
3.5.3	Core Layers	33
3.6	Pattern Library	35
3.6.1	Our Interaction Patterns	37
3.6.2	Our Visualization Patterns	37
3.7	System-Level and Algorithmic Validation	42
3.7.1	Validation Findings	42
3.7.2	Authoring Workflow Efficiency	43
3.7.3	Representative System Comparison	46
3.7.4	Quantitative System Metrics	47

4	Conclusions and Future Work	49
4.1	Summary of Contributions	49
4.2	Discussion and Implications	50
4.3	Future Work	50
5	Bibliography	53
A	Appendix	57
A	Pattern Catalog	57
A.1	Interaction Patterns (Procedural)	57
A.2	Visualization Patterns (Feedback)	58
B	Gamification Elements Setup Tutorials	59
B.1	Ghost Pattern	60
B.2	Arrow Pattern	60
B.3	Label Pattern	61
B.4	Progress Indicator Pattern	62
B.5	Motion Indicator Pattern	62
C	Interaction Pattern Setup Tutorials	63
C.1	Activate Action	63
C.2	Trajectory Action	64
C.3	Question Action	65
C.4	Choice Action	66
D	Algorithmic Heuristics and Runtime Manifest	68
D.1	Definition	68

List of Tables

2.1	Comparison of XR authoring approaches.	21
3.1	Gap-pattern-benefit mapping motivating the introduction of new interaction patterns.	28
3.2	Stylized dictionary of implemented visualization and interaction patterns with representative images and concise descriptions, including newly introduced procedural and motivational cues.	39
3.3	Mapping between Interaction Patterns (procedural actions) and Visualization Patterns (feedback cues). For thumbnails and references to the primary descriptions of each pattern, see Appendix A.	41
3.4	System-level validation summary.	48
A.1	Interaction Pattern catalog with representative thumbnails and provenance references.	58
A.2	Visualization/Gamification Pattern catalog with thumbnails and provenance references.	59

List of Figures

1.1	From declarative authoring to runtime gamified feedback: (a) Scene-graph overview, (b) Action Node setup, (c) Gamification Panel configuration, and (d) runtime execution with Ghost visualization. Arrows illustrate the transition from authoring to runtime.	4
2.1	Conceptual architecture of programmable reality. The paradigm shifts from traditional imperative scripting to declarative, pattern-driven authoring. Reusable data assets for gamification, interaction, and analytics are interpreted at runtime to generate adaptive, pedagogically coherent XR environments. Feedback from user actions continuously informs the authoring logic, forming a closed adaptive loop.	6
2.2	Illustrates the Virtuality Spectrum, showing how AR, MR, and VR form a continuum between physical and digital reality (adapted from (CreatXR, 2022).)	8
2.3	Dale’s Cone of Experience, illustrating how active engagement enhances retention (source: Growth Engineering) (Engineering, 2023).	9
2.4	Examples of imperative scripting approaches in XR authoring.	11
2.5	The Situated Visualization Patterns proposed by Lee et al. (2023)	13
2.6	Comparison of traditional visual scripting systems used in XR development: Unity Bolt (left) and Unreal Blueprints (right). Both provide low-code workflows but lack declarative, reusable feedback representations, motivating the need for a pattern-driven authoring approach.	15
2.7	Scenograph authoring interface: training scenarios represented as directed graphs of Action Nodes and flow nodes.	15
2.8	Configuration of a Use Action Node. The node combines logic, object bindings, success criteria, and feedback options.	16
2.9	Gamification Panel: no-code configuration of feedback patterns (glyphs, ghosts, labels, trajectories) attached to Action Nodes.	17
2.10	Gagné’s Nine Events of Instruction (Gagné, 1985), illustrating the sequential instructional process from attention and guidance to performance and retention. Sourced from (Growth Engineering).	19
3.1	Overview of the methodological process with provenance coding. Gray = pre-existing elements, dashed blue = extended components, green = new contributions introduced in this thesis.	23
3.2	Conceptual foundations of the proposed methodology showing how visualization, and interaction integrate into the programmable reality paradigm.	24

3.3	Gamification Panel interface for a selected Action Node (Use — “Pour Alcohol on Cotton Ball”). Authors can edit and configure Ghost and Arrow directly within the panel.	29
3.4	System architecture of the pattern-driven gamification framework. The authoring layer defines reusable interaction and visualization patterns, which are exported as JSON manifests. At runtime, the Heuristic Engine and Pattern Manager interpret these definitions to drive feedback and guidance within the XR training environment. .	32
3.5	Data model of the Gamification Subsystem showing relationships between <i>Scenegraph</i> , <i>ActionNode</i> , <i>GamificationProfile</i> , and associated pattern and heuristic classes.	32
3.6	Runtime rule binding as an Event–Condition–Interaction (ECA) pipeline: node events feed the Heuristic Engine, which evaluates conditions and triggers concrete pattern operations.	34
3.7	Visual representation of a declarative Gamification Profile for an <i>Insert</i> . Each rule follows an Event–Condition–Action schema linking interaction events (<code>OnStart</code> , <code>OnUpdate</code> , etc.) to pattern operations (e.g., <code>Ghost.Show()</code> , <code>Arrow.Show()</code> , <code>Label.Show()</code>).	35
3.8	Overview of visualization patterns used in the framework.	36
3.9	Mapping between visualization patterns and interaction patterns. .	37
3.10	Dynamic configuration of the Ghost pattern through the Gamification Panel.	43
3.11	Representative authoring workflow for configuring the Ghost pattern.	44
3.12	Authoring workflow for configuring the Arrow gamification element.	44
3.13	Authoring workflow for configuring the Label gamification element.	45
3.14	Manual Unity implementation of the Insert Interaction showing the complete C# scripting workflow (≈ 150 lines of code).	46
3.15	Scenegraph-based implementation of the Insert Interaction in the baseline framework by Zikas et al. (2023), configured through a compact 5-line JSON specification.	46
3.16	Declarative, no-code configuration of the Insert Interaction in the extended, pattern-driven framework developed in this thesis.	47
A.1	Example of a Ghost pattern previewing correct placement alignment.	60
A.2	Arrow pattern dynamically pointing to an off-screen object.	61
A.3	Label pattern showing contextual guidance at runtime.	61
A.4	Progress Indicator visualizing continuous task completion.	62
A.5	Motion Indicator illustrating correct rotational gesture.	63
A.6	Activate Action node configuration in the Scenegraph Editor. . . .	64
A.7	Trajectory Action node configuration in the Scenegraph Editor. . .	65
A.8	Question Action node configuration in the Scenegraph Editor. . . .	66
A.9	Choice Action node configuration in the Scenegraph Editor.	67

Chapter 1

Introduction

“Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem.”

— Christopher Alexander, *A Pattern Language* (1977)

This principle, first articulated by Christopher Alexander, captures a fundamental idea that extends beyond architecture and into digital design: the power of patterns to solve recurring problems systematically. In the context of Extended Reality (XR) training—spanning Virtual, Augmented, and Mixed Reality—this idea becomes particularly relevant. XR technologies are increasingly employed in domains such as healthcare, industrial operations, and emergency response, where they enable learners to rehearse complex or high-risk procedures in safe, controlled environments. However, the development of such applications remains challenging: it involves integrating heterogeneous technologies for interaction, visualization, and learning management, while ensuring pedagogical coherence, tasks that still demand considerable technical expertise and manual effort.

A central concept underpinning this thesis is the use of **patterns**—reusable design abstractions that capture recurrent solutions to interaction and visualization problems in XR training. Building on established software design principles (Gamma et al., 1994) and situated visualization taxonomies (Lee et al., 2023), this work formalizes two complementary categories: *procedural interaction patterns* and *spatial visualization patterns*. The former describe learner actions such as *Insert*, *Remove*, and *Use*, while the latter define how instructional cues and feedback appear in 3D space through structures like *Ghost*, *Label*, *Panel*, *Glyph*, *Virtual Mirror*, and *Trajectory*. Together, these patterns provide a unified vocabulary for XR authoring, allowing interaction logic and feedback behavior to be composed, reused, and adapted declaratively rather than hardcoded. In addition to consolidating existing vocabularies, the present work introduces several new procedural and visualization patterns—such as *Trajectory*, *Question*, *Activate*, *Choice Action*, and *Motion/Progress Indicator*—that extend the expressiveness of programmable-reality environments.

A key difficulty lies in how XR training scenarios are designed and maintained. Most existing authoring tools rely on ad hoc scripting and scene-based asset manipulation, offering little support for reusability or systematic adaptation. Each new scenario must be built largely from scratch, even when the same instructional logic or interaction types recur. In practice, developers must manually script every feedback element and bind it to corresponding scene objects and events, a time-consuming, error-prone process that inhibits reuse and consistency across scenarios.

Moreover, gamification, widely recognized for enhancing motivation and engagement in learning and training (Hamari et al., 2014; Deterding et al., 2011)—is typically implemented in an equally fragmented manner. Feedback cues such as arrows, labels, or holographic previews are hardcoded into scenes as isolated elements, making them difficult to update, reuse, or align with broader pedagogical goals.

The motivation for this research arises from the growing need for **reproducible, scalable, and pedagogically grounded XR authoring tools**. Current approaches treat interaction logic and feedback design as separate, often hardcoded processes—limiting reuse and increasing development cost. Instructors and content authors lack means to represent pedagogical intent directly in the authoring interface, resulting in inconsistent guidance and limited adaptivity across scenarios.

To structure this investigation, the following research questions were formulated:

- RQ1:** How can feedback, guidance, and motivational elements be formalized as reusable, pattern-based components within XR authoring systems?
- RQ2:** How can procedural interaction patterns and visualization patterns be unified into a coherent, programmable framework that supports pedagogically meaningful XR design?
- RQ3:** What are the algorithmic and authoring benefits, such as reusability, scalability, and efficiency, resulting from the integration of such a pattern-driven gamification framework?

While this thesis draws motivation from learning and training applications, its primary contribution lies at the **system and framework level**, not at the user-experience level. The work does *not* evaluate learner performance, cognitive outcomes, or usability directly. Instead, it focuses on the **architectural design, data modeling, and algorithmic integration** of a pattern-driven framework for programmable reality environments. A central goal of this research is to **accelerate the authoring process**, reducing manual scripting effort and enabling non-programmers to create gamified XR training scenarios efficiently through reusable, declarative components. This distinction situates the present work firmly within the domain of **design science and software engineering**, providing a foundation upon which future pedagogical and user-centered studies can build.

Building on this foundation, the thesis introduces the **Gamification Subsystem** as a core architectural innovation that operationalizes these goals. The subsystem extends the scenegraph-based authoring framework of Zikas et al. (2023) with a structured, pattern-based approach to XR content creation. It combines established software design principles with contemporary XR pattern vocabularies to create reusable, parameterized *Gamification Profiles* attached to Action Nodes. Through a dedicated *Gamification Panel*, authors configure these elements visually, transforming a code-intensive workflow into a structured, no-code process. Figure 1.1 illustrates this transition from declarative authoring to runtime execution of gamified feedback.

The contributions of this thesis are threefold:

1. **Pattern-Driven Authoring Methodology:** Defines a systematic, pattern-driven methodology for XR authoring that integrates procedural interaction and spatial visualization patterns within a unified authoring methodology. This methodology establishes a formal basis for representing feedback and

guidance as *declarative data entities* rather than hardcoded behaviors, enabling reusability and consistency across programmable reality environments.

2. **Gamification Subsystem Design and Implementation:** Implements the proposed methodology through a fully realized *Gamification Subsystem*, extending the scenegraph-based authoring framework of Zikas et al. (2023). This contribution is primarily architectural and algorithmic, focusing on system modularity, runtime determinism, and seamless integration with existing authoring pipelines.
3. **Technical and Architectural Validation:** Evaluates the proposed framework from a system-engineering perspective, focusing on its **authoring efficiency, reduction of manual scripting effort, and architectural extensibility**. The validation examines how the Gamification Subsystem integrates with the existing scenegraph-based authoring model to streamline the configuration of feedback and guidance behaviors through reusable *Gamification Profiles*. Results demonstrate that the framework enables a significantly **faster and easier authoring workflow** by replacing manual coding with a structured, **no-code visual process**, while maintaining modularity and scalability for future extensions within programmable reality environments.

In summary, this work redefines XR training authoring as a pattern-driven process. By embedding guidance and motivational structures directly into the scenegraph architecture, it transforms a code-intensive workflow into a visual, data-driven design environment—laying the foundation for scalable, pedagogically coherent, and reproducible immersive learning experiences.

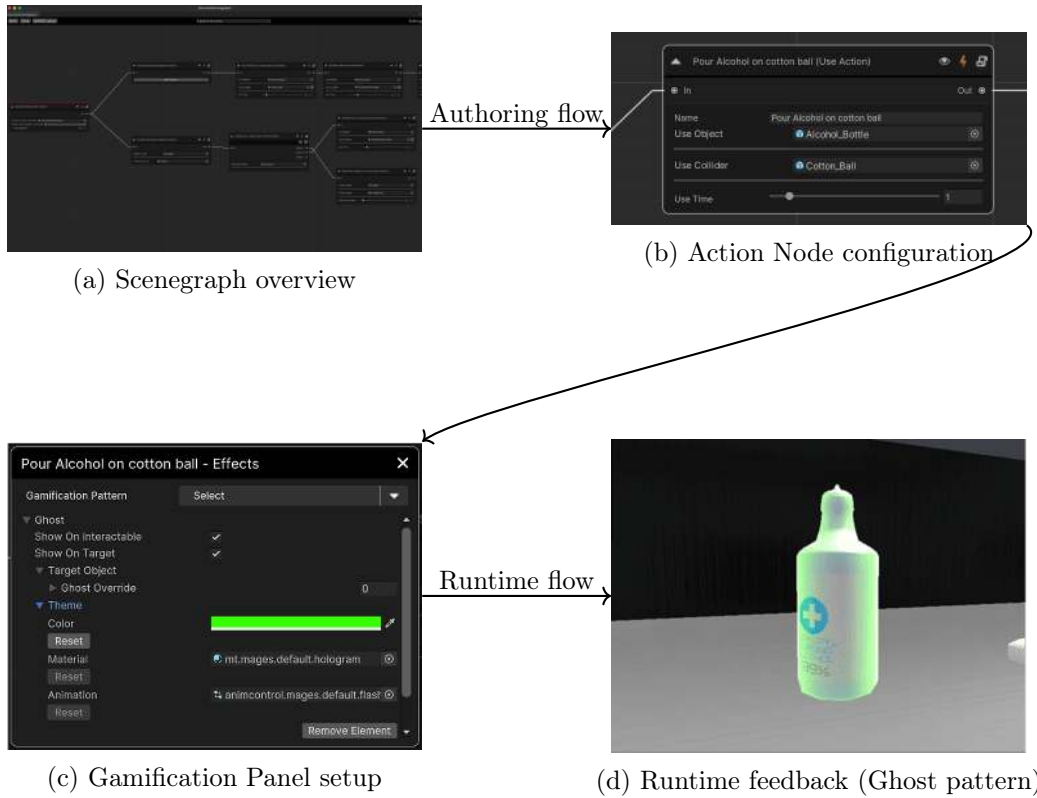


Figure 1.1: From declarative authoring to runtime gamified feedback: (a) Scene-graph overview, (b) Action Node setup, (c) Gamification Panel configuration, and (d) runtime execution with Ghost visualization. Arrows illustrate the transition from authoring to runtime.

Chapter 2

Background and Related Work

“We should build systems that describe what we want, not how to do it.”
— David Harel, *On Visual Formalisms* (1988)

Guided by Harel’s dictum, this chapter frames the thesis within the paradigm of **programmable reality** (Suzuki et al., 2025)—a declarative approach in which digital–physical experiences specify *what* pedagogical goals, interactions, and feedback behaviors should occur, while leaving *how* they execute to the runtime system. Within this paradigm, **Extended Reality (XR)**—encompassing Virtual, Augmented, and Mixed Reality (Radianti et al., 2022; Pottle, 2021)—serves as the experiential substrate for immersive learning, while **gamification** provides the motivational and structural layer that sustains engagement and organizes learning flow.

However, the authoring of such environments has traditionally been manual and fragmented: feedback cues are often hardcoded, interaction logic duplicated, and pedagogical intent rarely formalized. To address these challenges, this thesis proposes a **pattern-driven authoring framework** for programmable reality environments. The framework integrates **design patterns**, **scenograph architectures**, and **algorithmic heuristics** to automate feedback management, enhance reusability, and preserve pedagogical coherence.

The remainder of this chapter situates programmable reality within the broader landscape of XR, serious games, and gamification theory, tracing how these domains converge toward a shared need for reusable, data-driven authoring methods that enable scalable and pedagogically meaningful XR training design.

2.1 Programmable Reality

The concept of programmable reality was introduced by Suzuki et al. (2025) to describe a computational paradigm in which digital and physical environments are treated as a unified programmable continuum. Rather than defining interaction logic through imperative code, programmable reality employs declarative representations—data structures and rules that describe how perception, behavior, and feedback should dynamically adapt to user actions and contextual events.

Programmable reality aims to make immersive environments adaptive and composable: every element of the experience can be specified as data, enabling the runtime system to interpret and instantiate it dynamically. This approach contrasts with traditional XR development workflows, where feedback, logic, and gamification are often implemented as static, script-based components.

In practice, the framework supports systems that can adapt in real time, generate content procedurally, and deliver automated feedback. It replaces manual coding with declarative rules for interactions and visuals, letting designers focus on the user experience instead of technical details. This approach makes XR development more scalable and modular, with behaviors defined by patterns rather than fixed scripts.

Suzuki et al. (2025) outlined the conceptual basis, and recent work has translated this paradigm into practical XR systems. Building upon these foundational principles, the authoring framework (Zikas et al., 2023) extends the concept of programmable reality to the domain of XR training simulations. It provides a modular, pattern-driven authoring environment that unifies gamification, analytics, and spatial interaction through declarative configurations. Within this architecture, interactions and feedback are expressed as reusable data assets rather than fixed code. This thesis expanded that framework by formalizing a pattern-based methodology that transformed feedback, guidance, and motivational structures into composable, reusable components for XR training scenarios. Programmable reality thus provides the conceptual foundation for this thesis: a shift from ad-hoc scripting to reusable, data-driven structures that underpin the proposed pattern-driven gamification framework. Figure 2.1 visualizes this paradigm as applied in the present work, illustrating how declarative authoring, runtime interpretation, and adaptive XR feedback form a closed adaptive loop within a pattern-driven framework.

Although programmable reality defines a declarative paradigm for adaptive environments, its practical realization depends on the capabilities of Extended Reality (XR). The following section therefore examines how XR technologies enable experiential learning and provide the foundation on which programmable reality applications can be built.

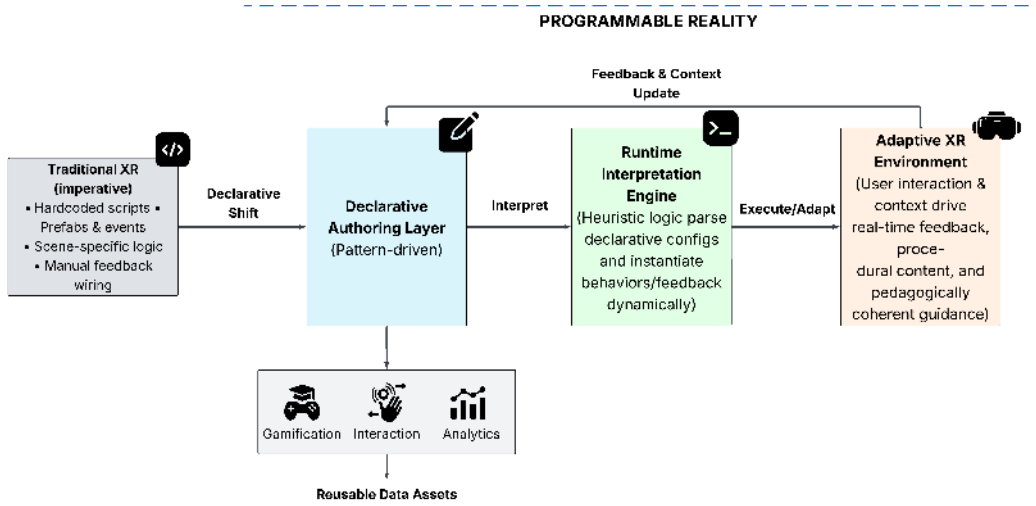


Figure 2.1: Conceptual architecture of programmable reality. The paradigm shifts from traditional imperative scripting to declarative, pattern-driven authoring. Reusable data assets for gamification, interaction, and analytics are interpreted at runtime to generate adaptive, pedagogically coherent XR environments. Feedback from user actions continuously informs the authoring logic, forming a closed adaptive loop.

2.2 Extended Reality and Serious Games

Extended Reality (XR) provides the technological foundation for the visualization patterns. **XR is an umbrella term encompassing Virtual Reality (VR), Augmented Reality (AR), and Mixed Reality (MR)**—technologies that integrate physical and digital elements in different ways (Radianti et al., 2022; Pottle, 2021). VR immerses users in fully virtual environments, AR overlays digital information onto the real world, and MR enables real-time interaction between physical and virtual objects. Once considered separate domains, these modalities are now unified under the broader concept of XR, often described along the reality–virtuality continuum (Milgram and Kishino, 1994), reflecting a shift toward cross-platform spatial experiences that operate seamlessly across devices and contexts.

Recent advances in hardware, such as the Meta Quest, Microsoft HoloLens, and Apple Vision Pro, have made MR particularly prominent by enabling spatial computing, where digital and physical elements coexist in the same perceptual space. In domains such as healthcare training, XR now enables realistic, hands-on, and risk-free practice.

Embodiment, the sense of presence, agency, and ownership of a virtual body, plays a key role in XR learning outcomes. Research shows that immersive VR enhances engagement, procedural recall, and motivation compared to traditional instruction (Kiltner et al., 2012; Hung et al., 2014; Ganier et al., 2014). By acting through a virtual body, learners form stronger cognitive and kinesthetic connections between perception and action, which in turn improves knowledge transfer to real-world contexts. For this reason, XR-based training has become a cornerstone of modern simulation-based learning, combining the safety of virtual environments with the authenticity of hands-on practice.

When used for education or professional training rather than entertainment, XR falls under the category of serious games—interactive systems designed primarily for learning rather than leisure. Serious games transform practice into structured learning experiences that promote retention, decision-making, and skill transfer.

The combination of XR and serious games holds great potential: learners can safely rehearse complex or high-risk tasks while receiving structured, adaptive feedback. Yet without clear guidance or progression, immersive environments risk overwhelming users with sensory input, leading to confusion or disengagement. Gamification addresses this challenge by providing the motivational and structural scaffolding that organizes feedback, sustains engagement, and transforms open-ended immersion into purposeful learning.

The relationship between Augmented, Mixed, and Virtual Reality is illustrated in Figure 2.2, which presents the Virtuality Spectrum adapted from CreatXR (2022). The figure situates AR, MR, and VR along a continuum between the physical and digital domains, emphasizing how they collectively form the foundation of Extended Reality (XR).

XR offers immersion and embodiment, yet immersion alone does not guarantee effective learning. The next section explores how **gamification** introduces the motivational and structural scaffolding that transforms open-ended interaction into purposeful training experiences.

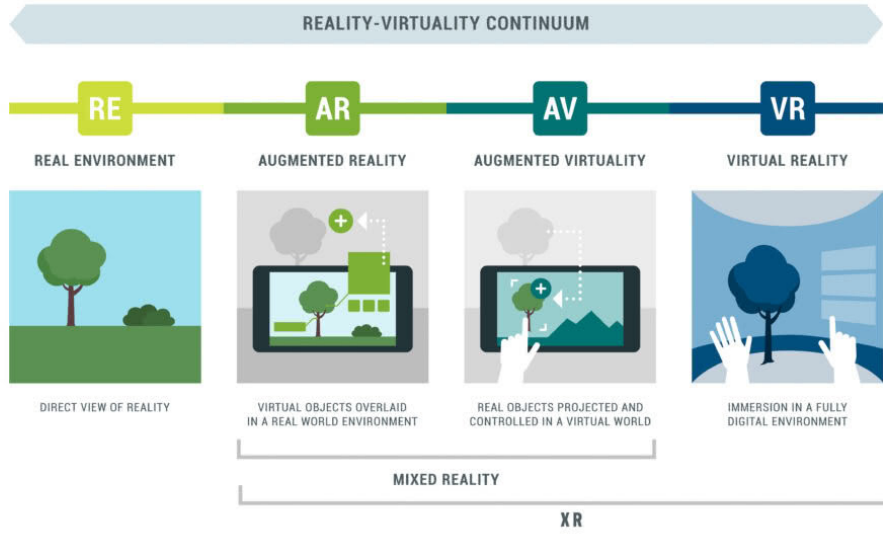


Figure 2.2: Illustrates the Virtuality Spectrum, showing how AR, MR, and VR form a continuum between physical and digital reality (adapted from (CreatXR, 2022).)

2.3 Gamification in Serious Games

Gamification refers to the use of game-derived elements, such as points, rewards, and progression systems, in contexts that are not primarily games. Its core principle is simple: game mechanics sustain motivation, focus, and persistence, standards that are equally valuable in education and professional training (Deterring et al., 2011; Werbach and Hunter, 2012; Kapp, 2012). By embedding clear goals, feedback loops, and incremental challenges, gamification transforms repetitive practice into a structured and engaging learning experience.

In serious games, the function of gamification differs from that in entertainment. While commercial games use mechanics mainly to create enjoyment, serious games apply them to support learning, guide attention, and provide immediate, informative feedback. A badge or score, for instance, does not merely reward performance, it signals the learner’s mastery of a specific skill and readiness to progress to more complex tasks. Through such mechanisms, gamification bridges the gap between knowledge acquisition and real-world skill transfer.

Aviation provides an illustrative example. No pilot takes control of a commercial aircraft without first mastering every procedure in a simulator. Modern flight instruction relies heavily on immersive and gamified environments that structure learning through repetition, performance tracking, and adaptive feedback. Airbus’ Virtual Procedure Trainer (VPT) and VRpilot’s VRflow system exemplify this approach. The VPT enables flight crews to rehearse cockpit procedures in interactive 3D environments, developing muscle memory and procedural fluency through step-wise guidance and real-time evaluation (Airbus, 2025). Similarly, VRflow integrates gamified feedback, scoring systems, and adaptive scenario progression for aircraft (VRpilot, 2025). These systems demonstrate how gamification converts repetition into measurable progression, improving engagement and long-term retention while maintaining safety.

Cognitive studies further support this effect. VR-assisted learning has been

shown to demonstrate substantial improvements in retention, often several times higher than traditional instruction (VRpilot, 2025). This aligns with Dale (1946)’s Cone of Experience (see Figure 2.3), which suggests that learners retain about 10 % of what they read but up to 90 % of what they do. By engaging learners through active, embodied practice, gamified VR training reinforces experiential learning, critical in domains where procedural accuracy and rapid decision making are essential.

When such gamified principles are embedded directly within immersive simulations, they form the basis of gamification in XR training.

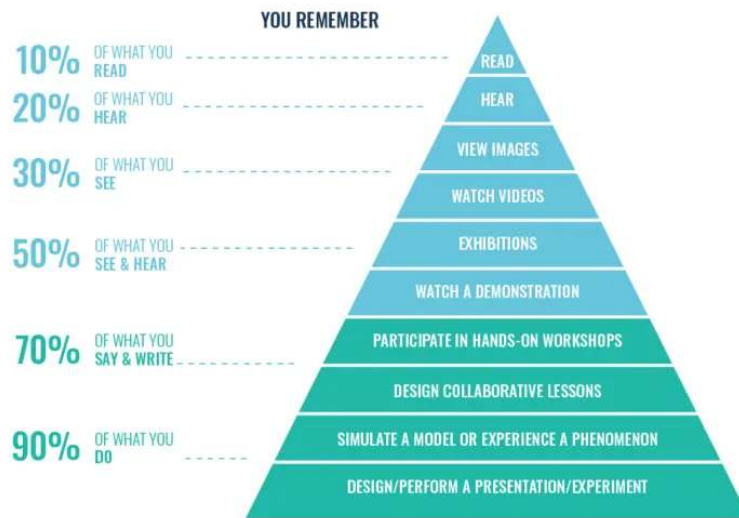


Figure 2.3: Dale’s Cone of Experience, illustrating how active engagement enhances retention (source: Growth Engineering) (Engineering, 2023).

Having established the motivational role of gamification in serious games, we now focus on its specific application in XR training. This transition highlights both the potential and the technical challenges of implementing adaptive, feedback-rich immersive instruction.

2.4 Gamification in XR Training

A primary challenge in immersive XR training is the potential for user disorientation arising from the absence of real-world instructional cues. In fully virtual environments, learners are separated from physical reference points, peer collaboration, and direct instructor guidance. As a result, they must rely entirely on in-simulation feedback to orient themselves, identify correct tools, and perform procedural steps in the proper sequence. Without adequate structure and guidance, users can easily become disoriented or frustrated, diminishing both engagement and learning effectiveness.

At this point, one might ask: if XR training can confuse learners, why use it at all instead of conventional methods? The answer lies in XR’s unique ability to deliver experiential learning. Unlike 2D or classroom-based instruction, XR allows participants to act and react within realistic, context-rich scenarios, learning by doing rather than by observing.

However, immersion alone does not ensure learning. Without structure, feedback, or motivation, even the most realistic environment can feel aimless. Gamification provides that missing structure. It functions as a motivational and organizational scaffold that turns raw immersion into purposeful, guided interaction. In well-designed XR training, gamified elements—such as arrows directing attention, holographic “ghosts” demonstrating correct actions, and panels providing contextual feedback—act as embedded instructors. They guide attention, clarify objectives, and deliver immediate feedback without breaking immersion.

For instance, in a medical training simulation, a learner might need to insert a catheter into the correct vessel. Without guidance, they could pick up the wrong tool or misplace it. By combining a guiding arrow, a holographic preview, and a contextual label, the system provides a layered scaffold that enhances both efficiency and confidence while minimizing errors.

Although gamified feedback enhances learning, most XR systems still depend on **manual scripting** to implement these behaviors. The next section examines why such workflows are unsustainable and introduces the idea of **algorithmic authoring** as a scalable alternative.

2.5 From Manual Scripting to Algorithmic Authoring

Building immersive training experiences in XR is not only about designing how things look and behave, but also about how efficiently those elements can be created, maintained, and reused. In this sense, the challenge is as much an engineering problem as it is a design one: how can we develop gamified XR training scenarios easily, quickly, and with minimal technical effort or coding expertise?

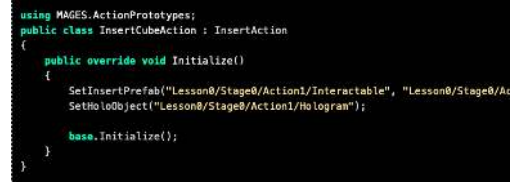
Traditionally, each interactive element, an arrow directing attention, a holographic preview illustrating correct action, or a label offering contextual feedback, must be manually scripted and bound to the precise moment it appears in a simulation. This process is slow, error-prone, and difficult to scale, since every feedback behavior must be reimplemented for each new training step. As training scenarios grow in complexity, the dual burden of implementation and design reasoning makes such solutions neither sustainable nor future-proof.

Recent authoring frameworks attempt to reduce this burden through no-code and low-code approaches (Zikas et al., 2023). These systems simplify configuration and lower the need for custom scripting, though pure no-code editors often remain constrained by predefined templates. A more scalable alternative is to move beyond manual scripting toward **algorithmic heuristics**, generalized rules (not user-experience heuristics) defining when and how feedback should appear, behave, and disappear. Instead of specifying each cue manually, developers describe reusable principles for how the system should respond to learner actions. Figure 2.4 illustrates the evolution from traditional manual scripting to low-code authoring approaches. The Unity example demonstrates how feedback behavior must be explicitly defined and linked through code, while the (Zikas et al., 2023) framework reduces this burden by encapsulating common interaction logic within predefined procedural methods. These heuristics do not focus on how the user feels or reacts, but on how the system manages its internal logic: when feedback should trigger, how elements remain consistent, and how similar behaviors can be reused across scenarios. By defining such rules algorithmically, the process of integrating gamification becomes faster, more reliable, and easier to scale across multiple XR training applications.

To translate these heuristics into practice, behavior must be represented as



(a) Manual scripting in Unity.



(b) Insert Action in (Zikas et al., 2023) framework.

Figure 2.4: Examples of imperative scripting approaches in XR authoring.

structured data rather than procedural code. This is achieved through two complementary paradigms: **node-based visual programming** and **scenegraph architectures**.

In node-based authoring, procedural logic is represented as a directed graph of nodes—discrete operations connected by control and data flows. This approach enhances composability, inspectability, and reusability. Authors can visually combine small behaviors, such as “Insert,” “Use,” or “Remove,” into complex procedures without rewriting code. Compared with block-based editors, node graphs are better suited to representing concurrency, many-to-many relations, and persistent state—patterns common in spatial and reactive XR simulations.

A scenegraph complements this paradigm by capturing the declarative state of a simulation: objects, relationships, and procedural steps. Each node in the graph represents an Action Node with references to scene entities, success conditions, transitions, and attached feedback definitions. Because nodes are treated as reusable data assets rather than monolithic scripts, they can be serialized, versioned, and programmatically transformed. This structure promotes separation of concerns, deterministic orchestration, and portability across projects and platforms.

Within this framework, gamified feedback is treated as structured data rather than hard-wired logic. Each Action Node autonomously manages its activation and completion events, defining when feedback appears, how success is detected, and when the system transitions to the next action. Heuristic rules, such as show an arrow when the target is off-screen, fade a ghost on alignment, or display a label on completion, are then expressed declaratively and executed uniformly by the runtime.

This approach transforms gamification from a handcrafted process into a scalable, algorithmically driven authoring paradigm. It allows XR training systems to move from individually scripted experiences toward reusable, data-defined interaction models, laying the foundation for adaptive, pattern-based design across future serious XR applications. Building on this fundamental shift, this thesis introduces a Gamification Subsystem that operationalizes algorithmic authoring for motivational and guidance feedback, demonstrating how such automation can enhance scalability, consistency, and pedagogical alignment in XR training.

2.6 Frameworks and Theoretical Basis

Reusable gamification in XR training is built on four complementary foundations. Each addresses a different aspect of how feedback is designed, delivered, and im-

plemented. Integrating gamification into XR training draws upon multiple theoretical and technical traditions. Four complementary foundations underpin this framework:

- **Gamification theory** — Explains the motivational logic behind cues and rewards. It defines why feedback elements such as goals, progress indicators, or rewards sustain engagement and structure task flow.
- **Visualization patterns** — Define how information appears in 3D space. Elements like arrows (glyphs), panels (labels), or holographic previews (ghosts) create a vocabulary for representing feedback that is clear, spatially aligned, and consistent.
- **Learning science** — Provides principles for when and how much information to display. Theories of cognitive load and embodied learning guide the timing, pacing, and intensity of feedback to support comprehension rather than distraction.
- **Technical architecture** — Describes how these ideas are implemented within a reusable framework. Rather than focusing on code, interactions and feedback are represented as structured data, enabling consistent and scalable authoring across XR scenarios.

2.6.1 Gamification Theory

Gamification theory helps explain how feedback, rewards, and challenges can sustain engagement when training tasks are repetitive or complex. Models by Werbach and Hunter (2012), Kapp (2012), and Molina-Carmona et al. (2020) describe three layers of gamification: dynamics (motivational drivers such as goals or narrative), mechanics (rules and feedback systems), and components (the concrete elements such as arrows, holograms, or labels). This layered perspective is essential in XR, where excessive visual cues may increase cognitive load. Gamification theory therefore acts as a filter, emphasizing that these patterns are not decorative but functional tools that align motivation with learning outcomes.

2.6.2 Situated Visualization Patterns

A second foundation is the concept of situated visualization. Lee et al. (2023) identify recurring patterns for AR, such as glyphs, panels, and ghosts (see Figure 2.5, that define how information can be embedded meaningfully in physical space. This thesis extends these visualization patterns with a gamification layer: glyphs become attention-directing arrows, panels deliver instructional cues, and ghosts demonstrate correct actions. In this way, visualization patterns evolve into **gamification patterns**, not only conveying information but guiding, motivating, and reinforcing learning.

2.6.3 Learning Science Foundations

Learning science provides the pedagogical foundation that ensures gamification enhances understanding instead of distracting from it. Cognitive Load Theory (Sweller, 1988; Sweller et al., 2011) shows that learners can only handle a limited amount of information at once. Research on embodied learning (Johnson-Glenberg, 2018) demonstrates that physical movement, gestures, and spatial interaction help strengthen memory and comprehension. Social learning theories (Vygotsky, 1978; Lave and

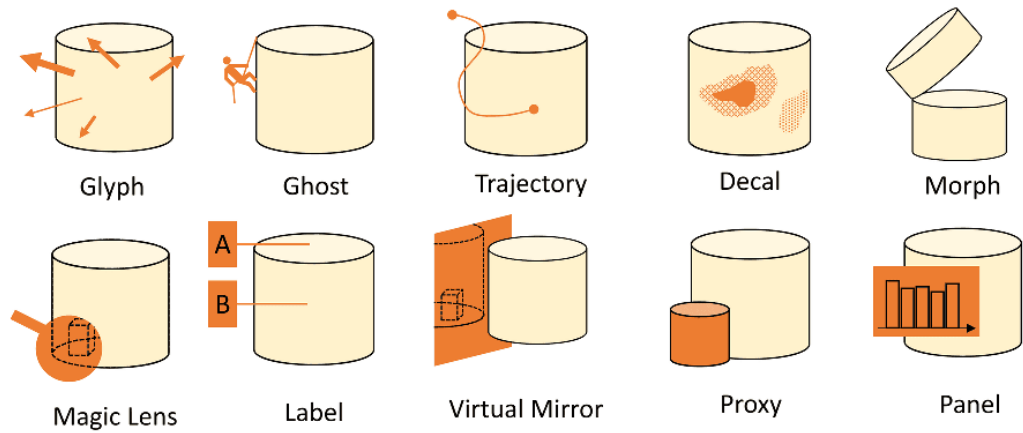


Figure 2.5: The Situated Visualization Patterns proposed by Lee et al. (2023)

Wenger, 1991) emphasize the value of scaffolding and collaboration in building skills. Together, these perspectives suggest that XR visualization patterns should provide just-in-time guidance, clear enough to support learners, but subtle enough to prevent cognitive overload.

2.6.4 The Pillars of Programmable Reality

Together, these four foundations form the conceptual basis of **programmable reality**. Gamification theory explains the motivational logic that sustains engagement and structures learner progression. Visualization patterns define how information is spatially represented and perceived within immersive environments. Learning science ensures that feedback mechanisms support comprehension and cognitive efficiency. Finally, the scenegraph architecture translates these theoretical principles into reusable, data-driven components for practical implementation.

Combined, these pillars enable XR training environments where feedback and guidance are not fixed scripts but modular, adaptive patterns that can be reused and extended across scenarios. This synthesis transforms XR gamification from a collection of isolated design choices into a structured, pattern-driven framework—one capable of supporting both human learners and, in the future, autonomous agents.

2.7 Design Patterns in XR and HCI

Originating in architecture where Alexander et al. (1977) described reusable spatial solutions, the design-pattern concept was later adopted in software engineering and HCI to formalize recurring solutions to technical and interaction problems.

This idea was later adopted in software engineering. The “Gang of Four” (GoF) book by Gamma et al. (1994) defined common programming structures like Observer, Factory, and Decorator, giving developers a shared vocabulary for addressing recurring technical challenges.

Human–Computer Interaction (HCI) researchers extended this perspective to interactive systems. Borchers (2001) introduced design patterns for interface design, describing solutions for common tasks such as navigation, user feedback, or dialog structure. The goal remained the same: to make design knowledge explicit, reusable, and communicable across projects (Dearden and Finlay, 2006; Tidwell, 2011).

In immersive technologies, this approach has also been applied to Extended Reality (XR). Lee et al. (2023) proposed ten situated visualization patterns for Augmented Reality, including Glyphs (directional indicators), Panels (annotations), and Ghosts (holographic previews). These patterns describe how digital information can be positioned in space so that it is perceptually clear and contextually meaningful.

This thesis builds on that tradition by combining visualization and interaction patterns within a programmable framework for XR training. It shows how design knowledge can be captured as reusable structures that speed up development and improve instructional consistency.

Later chapters take this further by combining visualization patterns with interaction patterns (e.g., Insert, Remove, Use, Trajectory) to form what this thesis terms **programmable reality**. In this framework, XR authoring moves beyond custom coding toward the composition of established, reusable design patterns—making immersive training scenarios faster to build, easier to maintain, and more pedagogically grounded.

While design patterns provide conceptual structure, their practical realization depends on authoring systems capable of expressing these reusable behaviors.

2.8 XR Authoring Tools and Visual Scripting

Developers typically create XR training scenarios using general-purpose game engines such as Unity or Unreal Engine. Both support immersive development through dedicated XR toolkits and visual scripting systems (e.g., Unity Bolt, Unreal Blueprints), which allow prototyping without writing raw code. As shown in Figure 2.6, these node-based tools visualize event logic and object interactions through connected graphs, but each cue or behavior must still be defined and wired manually. However, when it comes to implementing contextual feedback, such as directional arrows, proximity indicators, or success messages, these tools rely heavily on prefab-based and event-bound scripts. Each feedback element must be manually instantiated, linked to scene objects, and synchronized with triggers or animation events. Even minor changes, such as modifying the timing of a cue or adjusting its visual state, require re-editing node graphs or script components individually. Consequently, feedback logic remains tightly coupled to specific objects and scenarios, limiting reuse across training applications.

Alongside these engines, domain-specific authoring tools have emerged, such as 3D Organon for medical anatomy, SimX for clinical simulation, and toolkits like VRTK offer simplified authoring interfaces. These platforms accelerate scene assembly but still treat feedback as static, scene-bound assets. Visual cues are inserted manually and lack parameterized control over when and how they appear. Since behaviors are embedded in scene hierarchies or timelines rather than abstracted as reusable rules, instructors must recreate or reconfigure each feedback element for every new scenario.

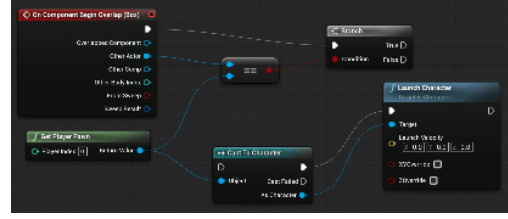
Across both categories, a common limitation remains: they separate content design (the narrative, environment, and task flow) from feedback design (the scaffolds, gamification, and guidance). As a result, feedback mechanisms are handcrafted and scenario-specific, inhibiting scalability and consistency.

To address these constraints, more structured authoring paradigms have begun to emerge. Among them, the scenegraph approach stands out: scenarios are represented not as linear scripts but as interconnected graphs of nodes. Each node encapsulates a discrete unit of interaction and can be configured visually, with

declarative links to feedback and success conditions. The following section explores how scenegraphs and interaction patterns establish a foundation for reusable, gamified XR training design.



(a) Unity Bolt visual scripting graph for an XR interaction. Each event, condition, and feedback must be connected manually, making reuse limited.



(b) Unreal Engine Blueprint implementing similar logic. Although node-based, feedback and interaction behaviors remain scenario-specific.

Figure 2.6: Comparison of traditional visual scripting systems used in XR development: Unity Bolt (left) and Unreal Blueprints (right). Both provide low-code workflows but lack declarative, reusable feedback representations, motivating the need for a pattern-driven authoring approach.

2.9 Scenegraph Architecture and Interaction Patterns

XR training scenarios are typically structured using a scenegraph architecture: a directed graph of nodes that defines the flow of a simulation. Rather than writing long procedural scripts, authors connect nodes visually, much like building a flowchart, as shown in Figure 2.7. This representation makes scenarios easier to design, debug, and reuse across different contexts.

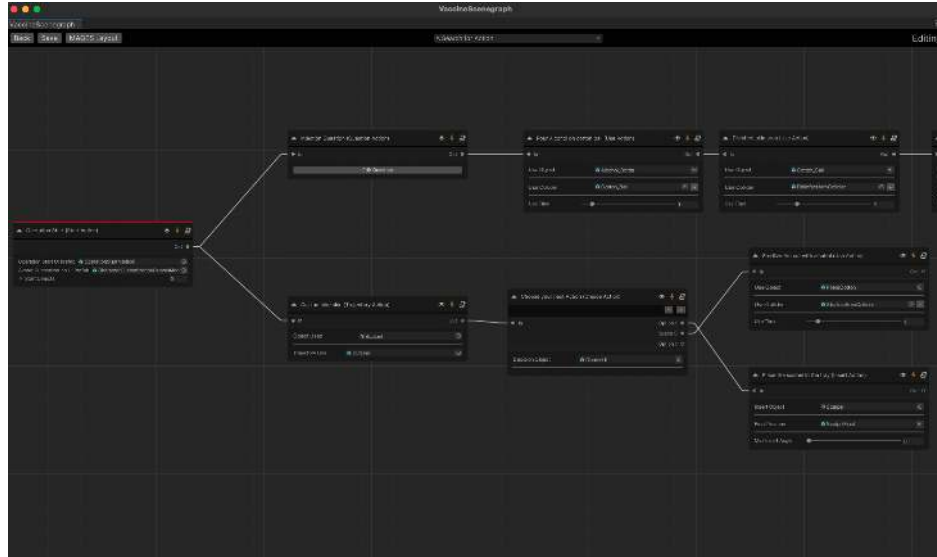


Figure 2.7: Scenegraph authoring interface: training scenarios represented as directed graphs of Action Nodes and flow nodes.

Each node in the graph represents an Interaction Pattern, encapsulating both the logic and the content of an interaction. Typical examples include Insert, Use, Remove, and Trajectory, while additional types such as Activate, Question, or Choice extend the system’s expressiveness. Flow nodes manage sequencing and

branching, allowing authors to define complex procedural structures visually.

Each Action Node specifies the object involved, the conditions for success, and the feedback shown to the learner. All of this is defined through a visual editor, no custom coding is required. Because nodes are modular data assets, they can be reused and adapted across training scenarios without duplicating logic. Figure 2.8 illustrates a representative Use Action Node configuration.

A key feature of this approach is that every Action Node includes default gamification presets. Instead of starting from a blank configuration, each action loads with research-informed feedback patterns suitable for its type. For instance, a **Use** action may include:

- a **ghost hologram** to preview correct placement,
- a **glyph arrow** to direct attention when the target is off-screen, and
- a **label** to provide contextual reinforcement.

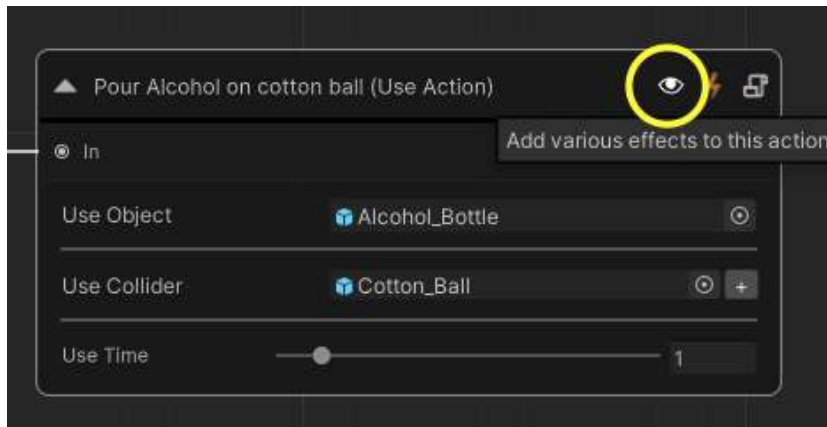


Figure 2.8: Configuration of a Use Action Node. The node combines logic, object bindings, success criteria, and feedback options.

Authors can refine or remove these defaults as needed, but the baseline ensures that effective scaffolding is always present from the outset. This approach saves time, reduces errors, and promotes consistency across XR training designs. The dedicated Gamification Panel shown in Figure 2.9 allows authors to attach and configure feedback patterns—such as glyphs, ghosts, and labels, directly to Action Nodes through a no-code interface. This integration ensures that gamification remains consistent and easily adjustable throughout the authoring process.

2.10 Motivation

Why is this work needed? Although XR training, gamification, and authoring tools have matured considerably, several challenges persist. This work is motivated by three key gaps in the current state of XR authoring:

- (a) **technical complexity**, which makes authoring dependent on programming expertise and hinders adoption by educators
- (b) **limited pedagogical integration**, where gamification and feedback are implemented as aesthetic or motivational add-ons rather than being aligned with learning objectives and

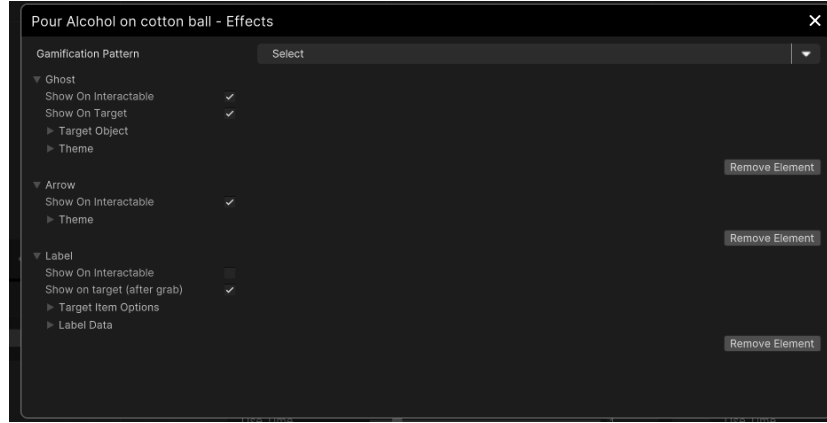


Figure 2.9: Gamification Panel: no-code configuration of feedback patterns (glyphs, ghosts, labels, trajectories) attached to Action Nodes.

- (c) **lack of reusable, systematic feedback mechanisms**, resulting in redundant development effort and inconsistent learner experiences across scenarios.

First, **authoring complexity gap**. Developing XR training scenarios typically involves combining scripts, prefabs, and external plugins. This process requires specialized technical skills that most educators and domain experts lack, creating a barrier to adoption and scalability.

Second, **pedagogical integration gap**. Many training applications follow rigid, step-by-step procedures with little feedback or adaptive guidance. Without meaningful scaffolding, learners often rely on trial and error, leading to frustration, cognitive fatigue, or disengagement (Queiroz et al., 2023; Hamari et al., 2014).

Third, **reusability gap**. While theoretical work on gamification and XR design patterns is well established (Deterding et al., 2011; Johnson-Glenberg, 2018; Sweller et al., 2011; Lee et al., 2023), few authoring systems make these concepts practical or reusable. Developers frequently re-implement the same feedback mechanisms from scratch in every new scenario.

Recent research therefore points to the need for authoring frameworks that embed visualization patterns directly into scenegraph-based architectures. Such systems should provide default, research-informed feedback presets that can be configured without coding, streamlining prototyping while promoting consistency and pedagogical reliability across XR training designs.

The reviewed foundations highlight the conceptual need for declarative, pattern-driven authoring in XR environments. The following sections build on this foundation by examining prior research in gamification, instructional design, visualization patterns, and scenegraph-based authoring systems—areas that directly informed the design of the proposed framework.

2.11 Prior Research and Related Work

“All scientific work is cumulative: it builds on what has gone before.”
— Robert K. Merton, *The Sociology of Science* (1973)

Building on the preceding theoretical foundations, this section reviews prior research in gamification, instructional design, XR visualization patterns, and authoring systems. It situates the present work within these domains, highlighting

the persistent gap between conceptual frameworks and reusable, system-level implementation. Specifically, it examines:

1. gamification and engagement in learning and training,
2. design patterns and instructional guidance in XR, and
3. scenegraph-based authoring frameworks that support modularity and reusability in immersive environments.

Across these domains, a consistent limitation emerges: gamified feedback is rarely implemented as a reusable, parameterized component that can be authored, configured, and reused systematically. Instead, feedback is often hardcoded through scripts or prefabricated assets, making it difficult to maintain consistency, scalability, or pedagogical alignment. This thesis addresses that gap by extending the authoring framework of Zikas et al. (2023) with a dedicated Gamification Subsystem that formalizes feedback behaviors as reusable, data-defined components.

2.12 Gamification in Learning and Training

Gamification—the application of game design elements such as rewards, progress indicators, and challenges in non-game contexts—has been widely studied for its effects on motivation and performance (Deterding et al., 2011; Hamari et al., 2014; Werbach and Hunter, 2012; Kapp, 2012). In learning and professional training, research consistently shows that structured feedback, clear goals, and immediate reinforcement improve engagement and long-term retention (Molina-Carmona et al., 2020).

In XR environments, gamification strengthens procedural learning through spatially embedded cues such as arrows, highlights, and holographic previews (Zarraoandía et al., 2019; Ketter et al., 2016). However, most studies evaluated learning outcomes rather than addressing implementation mechanisms within XR authoring systems. As a result, gamified feedback is usually scripted manually and lacks a standardized data model for reuse. The proposed Gamification Subsystem bridged this gap by embedding gamification logic directly within the authoring framework. It enabled authors to configure and reuse feedback profiles declaratively during authoring, transforming theoretical principles into a practical, reusable design mechanism.

While these studies robustly establish the *why* of gamification—its motivational and cognitive benefits—they offer limited insight into the *how*: the concrete methods by which gamification can be systematically authored, parameterized, and reused within XR environments. Most frameworks evaluate user outcomes but leave the underlying implementation ad hoc and scene-specific. This thesis addresses that gap by embedding gamification logic directly within the authoring framework, transforming theoretical efficacy into scalable, authorable practice.

2.13 Instructional and Cognitive Design in XR

Instructional theory provides the pedagogical foundation for effective feedback design. Frameworks such as Gagné’s Nine Events of Instruction (Gagné, 1985), Bloom’s Taxonomy (Bloom et al., 1956), and Merrill’s First Principles of Instruction (Merrill, 2002) emphasize scaffolding, feedback, and reinforcement as critical elements of learning. In XR training, these principles translate into spatial and

interactive guidance, such as visual cues or holographic demonstrations, that help learners maintain focus and confidence during complex procedures.



Figure 2.10: Gagné’s Nine Events of Instruction (Gagné, 1985), illustrating the sequential instructional process from attention and guidance to performance and retention. Sourced from (Growth Engineering).

Cognitive Load Theory (Sweller, 1988; Sweller et al., 2011) and embodied learning frameworks (Johnson-Glenberg, 2018) further stress the importance of presenting feedback at the right time and intensity to prevent overload. Despite this, most XR authoring systems do not operationalize instructional logic within their design architecture. The Gamification Subsystem extends this capability by integrating pedagogically informed heuristics into the underlying scenegraph model, ensuring that feedback and motivation are expressed consistently across scenarios. This integration ensures that pedagogical principles are not post hoc additions but embedded directly in the interaction logic of XR authoring.

Although these instructional theories offered strong pedagogical foundations, they were rarely operationalized within XR authoring tools. Existing systems tend to treat instructional scaffolding and feedback as external considerations rather than embedded design mechanisms. Consequently, there remains a disconnect between pedagogical models and the technical architectures used to realize them. The Gamification Subsystem bridges this gap by integrating pedagogically informed heuristics directly into the authoring process, ensuring that learning principles guide—not follow—the design of feedback behavior.

2.14 Design Patterns and Guidance in XR

Situated visualization patterns, such as those proposed by Lee et al. (2023), define reusable spatial structures for guiding user attention in XR environments. Patterns

like *Glyphs, Panels, Ghosts, etc*) describe how information should appear in space to remain perceptually clear and contextually meaningful. These contributions provide valuable representational guidance but do not address how such patterns can be parameterized, stored, or reused within authoring tools.

The present work extended that concept by embedding visualization patterns within the scenegraph-based authoring framework of Zikas et al. (2023), transforming them into programmable, reusable feedback structures. Through the proposed Gamification Subsystem, these patterns are not only represented visually but are also bound to procedural semantics, and controlled by algorithmic heuristics that define when and how they appear.

While situated visualization research offers valuable representational guidance, it seldom addresses the procedural or algorithmic dimension of feedback control. The result is a rich vocabulary of spatial cues but no systematic mechanism for managing their behavior across different learning contexts. This thesis extends these contributions by coupling visualization patterns with procedural semantics and heuristic control, enabling their reuse as dynamic gamification elements within a unified authoring framework.

2.15 Scenegraph Authoring and System Extensions

The authoring framework developed by Zikas et al. (2023) introduced a scenegraph-based architecture for XR training in which procedural steps are represented as Action Nodes that encapsulate interaction logic, object references, and success conditions within a reusable visual structure—improving modularity and scalability. However, the original framework focused primarily on procedural flow and content orchestration and lacked an integrated mechanism for representing gamification or instructional feedback.

This thesis extends that architecture with a **Gamification Subsystem** integrated directly into the scenegraph. For every Action Node, a linked *Gamification Profile* specifies when and how guidance and feedback are triggered throughout the node’s lifecycle. Through a visual Gamification Panel, authors configure patterns such as arrows, holographic ghosts, and progress indicators without coding. These configurations are stored declaratively and executed uniformly by the runtime, ensuring consistency and scalability across XR training applications.

Although the framework by Zikas et al. (2023) established a strong foundation for modular XR authoring, it focused primarily on procedural sequencing and interaction flow. It lacked a dedicated mechanism for representing motivational feedback or instructional guidance. This thesis builds directly upon that limitation, introducing a **Gamification Subsystem** that embeds reusable feedback profiles into the scenegraph itself, closing the gap between procedural design and pedagogical engagement.

2.16 Research Gap and Aim

Despite substantial progress in XR training, gamification, and authoring frameworks, important gaps remain at the system and algorithmic levels.

First, existing frameworks seldom integrate gamification as a reusable, authorable subsystem. Most authoring tools still treat feedback elements as ad hoc visual assets rather than structured, parameterized data objects. This makes gamification time-consuming to implement and difficult to maintain across scenarios.

Second, prior research has explored visualization patterns (Lee et al., 2023) and interaction patterns (Zikas et al., 2023) largely in isolation. While visualization patterns describe how information appears in space and interaction patterns define how users perform tasks, the absence of integration between these perspectives limits the potential for systematic, pattern-based authoring in XR training. To contextualize these limitations, Table 3.1 summarizes representative XR authoring approaches along four system-level dimensions: the presence of reusable modules, binding of feedback to procedural logic, support for visual configuration, and the use of declarative data models.

Approach	Reusable Modules	Action-bound Feedback	No/Low-code Config.
Unity / Unreal Engine	◦ Prefabs	◦ Manual scripting	◦ Partial
No-code platforms (SimX, 3D Organon)	● Templates	◦ Limited binding	● Yes
Scenograph authoring (Zikas et al., 2023)	● Action nodes	● Procedural flow	◦ Limited feedback config.
This thesis (Gamification Subsystem)	● Gamification profiles	● Lifecycle-bound	● Visual configuration panel

Legend: ● = full/strong support, ◦ = partial or limited support.

Table 2.1: Comparison of representative XR authoring approaches across system-level dimensions: modularity, procedural feedback binding, and configuration flexibility.

This thesis addressed these gaps by introducing a unified, pattern-driven framework that extended the scenograph-based authoring model of Zikas et al. (2023) with a dedicated Gamification Subsystem. The subsystem uniquely combines the situated visualization patterns of Lee et al. (2023) with the procedural interaction structures of Zikas et al. (2023) into a single, coherent authoring environment. To the best of the author’s knowledge, this represents the first integration of these two paradigms within an operational XR authoring system.

Through this integration, visualization and interaction patterns are fused into reusable, algorithmically defined components that can be configured directly within the authoring interface. The overall aim is to operationalize gamification as a system-level design mechanism, transforming pedagogical and motivational principles into programmable, reusable structures for XR training environments.

Chapter 3

Methodology

3.1 Overview

“Design is not just what it looks like and feels like. Design is how it works.”

— Steve Jobs

Building XR training environments traditionally began with an empty scene: interactions were hand-coded, feedback was hardwired, and gamification—if included—was added in an ad hoc manner. This makes development slower, harder to maintain, and less consistent between training scenarios. The methodology in this thesis defines guidance and motivation as reusable patterns that are created through declarative authoring rather than manual scripting. The aim was a system-level methodological contribution, the development of abstractions, data models, and workflows that improved authoring speed, maintainability, and pedagogical consistency in XR training environments.

This thesis treated guidance and motivation as reusable design patterns embedded directly into the authoring process. In XR training, guidance gives clear visual and spatial cues for what to do next, while motivation keeps users engaged through feedback, progress tracking, and rewards. Each pattern encompasses **three inter-related dimensions** of experiential learning: the learner’s actions (interaction), perceptions (situated visualization), and the rationale for feedback (gamification logic) Deterding et al. (2011); Hamari et al. (2014); Zarraonandia et al. (2019); Molina-Carmona et al. (2020). This work integrates visualization patterns from Lee et al. (2023) with procedural interaction structures from prior XR pattern-based training frameworks Zikas et al. (2023), enabling authors to construct complete learning sequences visually rather than through custom code.

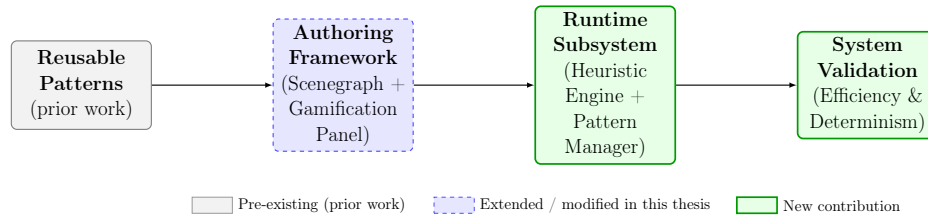


Figure 3.1: Overview of the methodological process with provenance coding. Gray = pre-existing elements, dashed blue = extended components, green = new contributions introduced in this thesis.

The proposed approach introduces a **pattern-driven authoring model for programmable reality** implemented as a module within a scenegraph-based

Unity editor. Authors design training flows through Interaction Patterns (*e.g.*, *Insert*, *Use*, *Remove*), each exposing a Gamification Panel that configures both (i) **visualization patterns** (*e.g.*, *Ghost*, *Arrow*, *Label*, *Trajectory Line*) and (ii) **procedural interaction patterns**, including the *Trajectory*, *Activate*, *Choice*, and *Question*. Feedback and guidance behaviors are managed automatically through *algorithmic heuristics*, simple runtime rules that determine when and how each visualization pattern activates. These heuristics connect author-defined data from the Gamification Panel with lifecycle events from each Action Node (*OnStart*, *OnGrab*, *OnAlign*, *OnSuccess*), allowing feedback to be fully declarative and deterministic.

To conclude, the methodology extends the concept of programmable reality by integrating interaction and visualization patterns into a unified authoring pipeline: (i) represent patterns as configurable components, (ii) expose them via no-code panels for each Interaction Pattern, (iii) apply evidence-based defaults, and (iv) execute them at runtime through serialized manifests. As a result, gamification became a systematic and reproducible element within the XR training authoring framework. Unlike the low-code model of Zikas et al. (2023), the proposed subsystem enables a **no-code** workflow for feedback: authors specify behaviors entirely as data (Gamification Profiles) via the Gamification Panel; no user-authored scripts or prefab hookups are required at any step.

3.2 Conceptual Foundations

The design of the proposed framework was grounded in the principles of **pattern-based design for immersive learning**. Rather than treating interaction and visualization as isolated or improvised features, the framework conceptualizes them as **reusable design patterns**—recurring solutions to common teaching and perception challenges in XR training. This approach is structured around three complementary pillars: *situated visualization patterns*, *interaction archetypes*, and *gamification as motivational scaffolding*.

Each of these pillars addresses a distinct dimension of the learning experience: situated visualization patterns guide *how learners perceive and interpret spatial information*; interaction archetypes define *how they act upon it*; and gamification mechanisms determine *how engagement and feedback are sustained throughout the task*. The following subsections outline these foundations and their role in shaping the overall design model.

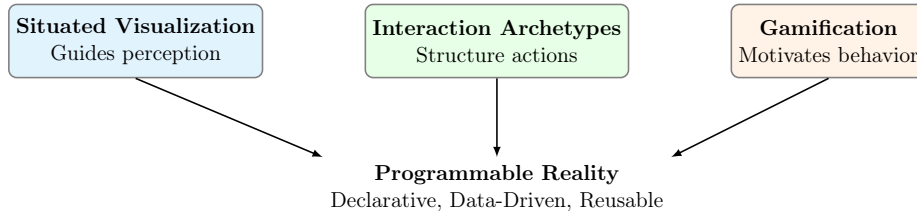


Figure 3.2: Conceptual foundations of the proposed methodology showing how visualization, and interaction integrate into the programmable reality paradigm.

Situated Visualization Patterns. Building upon the work of Lee et al. (2023), situated visualization patterns describe how visual cues and data overlays can guide user perception and understanding directly within three-dimensional space. Examples include the **Ghost** pattern, which previews a target state; the **Glyph**, which

highlights regions of interest; and the **Panel**, which presents contextual information. These elements provide spatial guidance by making it visually explicit what actions to take and where attention should be directed.

Interaction Archetypes. In parallel, prior research on XR authoring frameworks (Zikas et al., 2023) introduced a set of procedural and behavioral interaction patterns—such as *Insert*, *Use*, *Remove*, and *Activate*—that organize training tasks into reusable procedural units within a scenegraph. Each archetype defines a type of learner action, including its triggers, success conditions, and object relationships. Together, these patterns establish the procedural backbone of the framework, structuring how learners interact with the environment and how these actions are represented in the authoring interface.

Gamification and Motivational Scaffolding. While visualization patterns provide perceptual guidance and interaction patterns structure procedural behavior, gamification introduces a motivational layer that links learner performance to meaningful feedback. Drawing on the principles of Deterding et al. (2011) and Hamari et al. (2014), gamification in this context functions not as superficial reward mechanics, but as an integrated system that reinforces engagement and communicates progress. Through elements such as *Labels*, *Glyphs*, *Ghosts*, feedback and motivation are expressed through a consistent visual language that maintains instructional coherence across training scenarios.

Integration into Programmable Reality. *Programmable Reality* represents an authoring paradigm in which spatial interactions, visual guidance, and motivational feedback are treated as **declarative, data-driven, and reusable** components. By expressing these behaviors as parameterized data rather than hard-coded scripts, the framework enables **systematic, transparent, and scalable** authoring of immersive training simulations.

The paradigm is characterized by the following properties:

- **Declarative:** Desired behaviors—what to show, when to guide, how to confirm—are specified as structured data and rules rather than imperative logic.
- **Data-driven:** Configurations are serialized (e.g., manifests or profiles) and interpreted at runtime, ensuring portability and reproducibility.
- **Reusable:** Components (e.g., *Ghost*, *Arrow*, *Label*) and procedures (*Insert*, *Use*, *Remove*, *Activate*, *Trajectory*, *Question*, *Choice*) are parameterized and composable across scenarios.
- **Scenegraph-integrated:** Components are attached directly to Action Nodes within the scenegraph, preserving pedagogical intent and procedural coherence.
- **Heuristic-controlled:** The timing and management of feedback are automated through *algorithmic heuristics*—generalized rules that determine how the system reacts to learner actions and context.

This integration allows authors to construct complete learning experiences from modular, evidence-based building blocks while preserving instructional consistency. Beyond previously established visualization and interaction patterns, the introduction of *Activate*, *Trajectory*, *Question*, and *Choice* procedures expands coverage

to continuous motion, sequential activations, cognitive checkpoints, and branching decisions.

Together, these conceptual foundations provide the theoretical rationale for the methodological process that follows, bridging established models of learning-by-doing with their operational implementation in the XR authoring framework.

3.3 Methodological Approach

Each Interaction Pattern in the framework serves as a procedural container that defines what the learner does, while its associated Visualization Patterns define how feedback and motivation are expressed throughout that action’s lifecycle. This one-to-one binding ensures that every procedural step carries its own embedded guidance and motivational cues (*Ghost, Arrow, Label, Trajectory Line, Progress and Motion Indicators*), forming a unified pattern structure that governs both user behavior and feedback timing.

This methodology focuses on the implementation of reusable visualization patterns embedded into a functional XR authoring framework. The goal was to design, develop, and validate a system capable of integrating guidance and motivational feedback directly into the authoring workflow, transforming abstract gamification concepts into concrete, configurable, scalable modules.

The methodological focus was threefold:

- **Integrated gamification design:** embedding the gamification subsystem on the XR authoring framework and linking it directly to the scenegraph architecture. Each Action Node (*e.g., Insert, Use, Remove*) of the scenegraph contains a default set of gamification elements, such as *Ghost, Glyph, Label*, selected to provide meaningful feedback and added value based on the specific learning intent and interaction needs of that action. For example, insertion actions emphasize spatial alignment through a Ghost preview, while use actions highlight targets with a directional Arrow or Label.
- **Configurable and extensible authoring:** allowing developers to customize, remove, or add gamification elements through a dedicated Gamification Panel. This visual interface enables both per-action and global configuration, letting authors edit gamification parameters within a single node or apply consistent feedback patterns across the entire simulation. All modifications occur without scripting, making pattern-level design adaptable to different training contexts and scenarios.
- **Design–runtime coherence:** maintaining a direct correspondence between what authors configure in the editor and what learners experience in the simulation. All gamification settings were serialized with their parent nodes and dynamically instantiated at runtime. This ensured that design-time intentions, such as visual guidance or motivational feedback, translated faithfully into consistent learner experiences during execution.

The validation focuses on verifying the correctness, efficiency, and authoring usability of the proposed system. Specifically, it examines (i) architectural consistency between the authoring and runtime layers, (ii) performance and synchronization of gamification components, and (iii) authoring efficiency improvements compared to traditional scripting-based workflows.

3.4 Development Process

The development process followed a structured design implementation path that turned conceptual design patterns into functional and reusable templates for XR training authoring. Each phase built on the previous one to integrate, test, and document the visualization patterns within an XR authoring framework.

1. **Pattern Extraction and Analysis.** Both visualization and interaction patterns were systematically reviewed and distilled from previous research, visualization patterns from Lee et al. (2023), including *Glyph*, *Ghost*, *Panel*, *Virtual Mirror* and *Trajectory*, and interaction patterns from Zikas et al. (2023). Previously undocumented or partially implemented prototypes from earlier research, including those originally designed for related environments such as AR, were stabilized, parameterized, and extended into fully functional components suitable for XR contexts.

At this stage, each pattern was analyzed for both its teaching purpose and its structural fit within the XR authoring workflow. The study explored how visualization and interaction patterns could be connected, identifying when direct integration made sense (for example, linking a Ghost preview with an Insert action) and when a pattern should stay as a separate gamification element.

The goal was to make XR development easier and more efficient: to allow authors to build training scenarios without worrying about configuring multiple parameters, writing repetitive scripts, or manually linking feedback components. During this analysis, several gaps were identified in the existing interaction vocabulary, particularly for actions involving sequential triggers, continuous motion, and decision-driven procedures. While the core procedural patterns (*Insert*, *Use*, *Remove*) covered most manipulation and assembly tasks, they proved insufficient to represent the broader range of gamified behaviors observed during training design.

To address these limitations, four new interaction patterns—**Activate**, **Trajectory**, **Question**, and **Choice**—were introduced, each extending the expressive range of the existing framework. Each pattern emerged from specific design challenges encountered while developing gamified XR training modules. Repetitive trigger-based actions (e.g., spraying disinfectant) informed the Activate, whereas the need to represent continuous motion, such as cutting along a defined path, gave rise to the Trajectory. Similarly, the integration of cognitive checkpoints and branching logic in procedural workflows led to the Question, and Choice patterns.

While these interaction patterns emerged from technical and procedural analysis, their origin was rooted in gamification design requirements. During the development of the Gamification Subsystem, several instructional and motivational scenarios revealed gaps in the existing action vocabulary. Certain types of feedback—such as repeated visual confirmation, guided motion, and cognitive assessment—could not be adequately represented by the initial Insert/Use/Remove structure. To support these feedback behaviors declaratively, new interaction patterns were required, each introducing a procedural foundation that the corresponding gamification elements could attach to. For instance, the *Activate* pattern formalized repetitive trigger-based interactions needed for progress feedback, while the *Trajectory* pattern enabled continuous guidance for path-following tasks. Similarly, *Question* and *Choice* patterns

Gap Observed	New Interaction Pattern	Representative Scenario and Benefit
Repeated trigger actions	Activate	Pumping or spraying disinfectant; encapsulates activation state and success logic, eliminating ad-hoc counters or timers.
Continuous guided motion	Trajectory	Cutting or tracing along a path; exposes spline progress and tolerance for precise motion tracking.
In-situ knowledge checks	Question	Embeds short assessments directly into the procedural flow, verifying understanding without interrupting immersion.
Branching procedures	Choice	Allows selection between alternative task paths, formalizing branching logic and outcome binding in the scenegraph.

Table 3.1: Gap–pattern–benefit mapping motivating the introduction of new interaction patterns.

allowed cognitive and motivational feedback (e.g., quiz responses, branching outcomes) to be expressed directly within the scenegraph. To summarize how these extensions emerged from practical authoring demands, Table 3.1 outlines the identified design gaps, the corresponding new interaction patterns, and their benefits within the gamified XR authoring workflow.

These additions operationalize **RQ2** by extending the unified pattern language to cover trigger-based, continuous, evaluative, and branching behaviors that were not expressible with Insert/Use/Remove alone.

2. **Pattern Translation to Framework Components.** Before implementation, each visualization pattern was conceptually designed to define how it should behave and appear within immersive XR environments. This step included VR, AR, and MR to ensure that all components worked consistently across different levels of immersion, producing flexible elements that perform reliably in any setting.

Following the definition of their visual behavior and interaction logic, the patterns were realized as modular components in Unity within the XR authoring framework. Each visualization pattern, was developed as a self contained C# module encapsulating its rendering logic, and event-driven activation rules. Initially, these components were placed manually in test scenarios to observe their runtime behavior and to validate that they maintained visual and functional coherence across different actions and scenes.

After verification, the gamification components were integrated with the scenegraph system so that they could be automatically attached to each Interaction Pattern (e.g., *Insert*, *Use*, *Remove*) without requiring manual setup or repetitive code. This integration ensured that every learner action within the training flow would be accompanied by immediate guidance or motivational feedback by default, such as an Arrow highlighting a target, or a Ghost previewing the correct placement. By embedding gamification elements di-

rectly into the nodes that define user interactions, the framework guarantees consistent feedback behavior across all procedural steps, while dramatically reducing the authoring workload. Authors no longer need to manually link prefabs for each task, instead, gamification becomes an inherent part of the scenegraph’s logic.

Finally, the components were connected to a dedicated Gamification Panel, a visual configuration interface embedded in each Action Node. Developers can add, remove, or edit gamification elements either per action or across the entire simulation, adjusting parameters such as visibility, color or animation, as shown in Figure 3.3.

The translation of patterns into modular components and their attachment to Action Nodes address **RQ1** by formalizing feedback and guidance as reusable, declarative system entities.

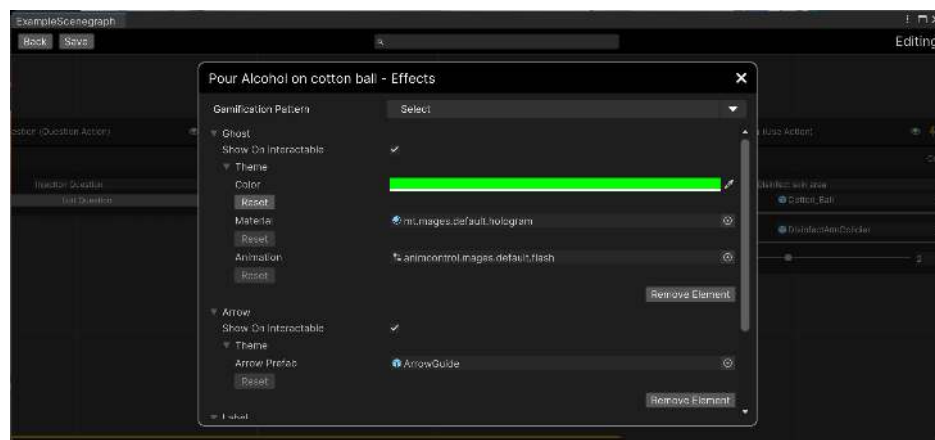


Figure 3.3: Gamification Panel interface for a selected Action Node (Use — “Pour Alcohol on Cotton Ball”). Authors can edit and configure Ghost and Arrow directly within the panel.

3. **Integration and Default Mapping.** Once the individual gamification components were validated, they were integrated into the authoring workflow through a system of default mappings. Each Action Node within the scenegraph automatically includes a predefined set of gamification elements suited to its instructional and motivational intent.

The default mappings between **Interaction Patterns** and **Visualization Patterns** were not assigned arbitrarily but derived from pedagogical and perceptual principles established in learning science and visualization research. Each pairing reflects the cognitive function required at that stage of the task: how to attract attention, clarify objectives, and reinforce success. For example, **Insert** actions employ a combination of *Ghost*, *Label*, and *Arrow patterns*, Ghosts preview correct placement to reduce spatial uncertainty, Labels provide concise procedural guidance, and Arrows ensure spatial orientation when targets move outside the learner’s field of view.

These associations were defined through iterative design analysis of common training procedures and validated against cognitive-load and scaffolding theory: high-perceptual-load steps receive more multimodal cues, while simple or familiar actions rely on minimal, confirmatory feedback. The resulting mappings, summarized in Table 3.3, constitute the framework’s default con-

figuration, ensuring that every procedural step carries pedagogically coherent guidance and motivational reinforcement by design.

Although default mappings are generated automatically, authors retain full creative control through the Gamification Panel. They can extend, remove, or modify elements to tailor guidance and feedback for specific training needs, unique scenarios, or different levels of learner expertise. In this way, the framework adapts to cases that go beyond the predefined defaults or require specialized instructional behavior.

The default mappings and lifecycle bindings further substantiate **RQ1**, ensuring that pattern behavior is instantiated consistently and reproducibly across procedures.

Derivation of Default Mappings. The default pairings between Interaction and Visualization Patterns emerged from an iterative synthesis of three evidence sources: (i) empirical observation of recurring training tasks, (ii) established instructional design models, and (iii) perceptual principles from situated visualization research.

- (i) **Task analysis:** Each procedural pattern was examined in representative XR training scenarios (e.g., assembly, disinfection, equipment operation) to identify its primary learner challenges—spatial precision, sequencing, or comprehension.
- (ii) **Pedagogical alignment:** Guidance mechanisms were then matched to these needs using scaffolding and cognitive load frameworks. Interactions requiring high spatial precision (e.g., *Insert*) were paired with concrete visual previews (*Ghosts*) to offload spatial reasoning; tasks demanding decision-making (*Question*, *Choice*) were linked with Labels to externalize cognitive processing and reinforce feedback loops.
- (iii) **Perceptual validation:** Finally, pairings were reviewed against situated visualization principles (Lee et al., 2023), ensuring that attention-directing cues (Arrows, Glyphs) complement rather than compete with contextual information.

Through this triangulated process, each mapping reflects not only functional convenience but also an instructional rationale: visual patterns minimize cognitive overhead, motivational cues sustain engagement, and together they create a coherent scaffolding layer that generalizes across XR training contexts.

4. **Iterative Refinement.** This gamification module underwent several refinement cycles focused on stability, authoring clarity, and development efficiency. Each iteration included internal authoring sessions where developers verified that default mappings, visual feedback, and runtime behavior worked as intended across all Interaction types. These sessions also evaluated development time, confirming that the integration of gamification did not add unnecessary complexity to the workflow but instead reduced repetitive setup and scripting compared to earlier XR development practices.

The refinement cycles further ensured that the mapping between user actions and their corresponding gamification responses was accurate and pedagogically meaningful. Each XR training scenario provides a coherent and

engaging experience, where every learner action is met with appropriate visual or motivational feedback, without requiring developers to manually adjust parameters or reconfigure feedback logic. At runtime, all gamification components were verified to remain visually and temporally synchronized, maintaining full immersion and consistency across the training flow.

These iterations also focused on accessibility for authors: ensuring that each gamification element is easy to understand, modify, or extend directly within the authoring environment. Developers can adjust feedback elements or create new combinations without needing advanced coding skills, extensive UX knowledge, or deep familiarity with gamification theory. Through this process, the system evolved from a functional prototype into a consistent, accessible, and pedagogically grounded authoring environment.

3.5 System Architecture and Implementation

The proposed framework extends a scenegraph-based XR authoring environment with an integrated gamification subsystem, creating a unified model for interaction design and motivational feedback. Authors begin by creating a *Scenegraph*, a visual representation of the training flow, composed of interconnected Action Nodes, each representing a distinct learner action or behavioral task. This node-based structure simplifies the creation of complex XR scenarios by allowing authors to define procedural logic and object interactions through an intuitive drag-and-drop interface, rather than writing thousands of lines of code.

3.5.1 Relation to the Underlying Scenegraph Framework

The proposed Gamification Subsystem extends, rather than replaces, the scenegraph-based authoring architecture of Zikas et al. (2023). The original framework defines the *Scenegraph* hierarchy, *Action Nodes*, and event-driven execution model that organize XR interactions. This thesis builds upon these core structures by embedding a new layer of functionality for gamification and feedback management directly within the Action Node architecture. As illustrated in Figure 3.4, the **Gamification Panel** is implemented as an integrated extension of the Action Node interface, allowing authors to configure *Gamification Profiles* visually without altering the existing scenegraph workflow. These profiles are bound to Action Nodes and processed by a newly introduced *heuristic engine*, which interprets declarative configurations and orchestrates runtime feedback behavior.

3.5.2 Gamification Subsystem Data Model

To formalize the internal structure of the subsystem, Figure 3.5 presents the data model underlying the Gamification Profile and Heuristic Engine. This model defines the relationships between the Scenegraph, Action Nodes, Gamification Profiles, and pattern instances, highlighting the composition and inheritance hierarchy that supports declarative configuration and runtime automation.

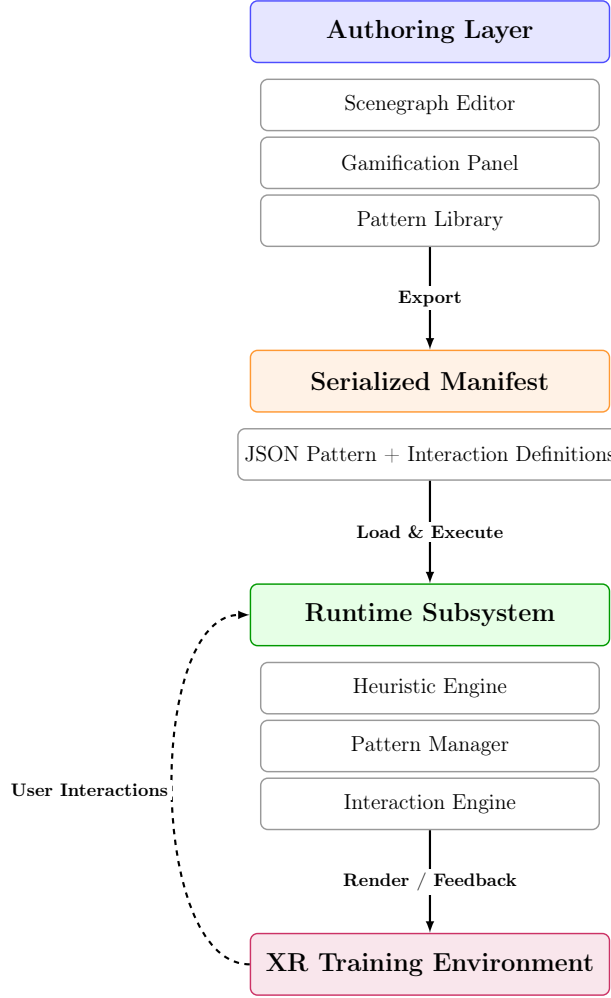


Figure 3.4: System architecture of the pattern-driven gamification framework. The authoring layer defines reusable interaction and visualization patterns, which are exported as JSON manifests. At runtime, the Heuristic Engine and Pattern Manager interpret these definitions to drive feedback and guidance within the XR training environment.

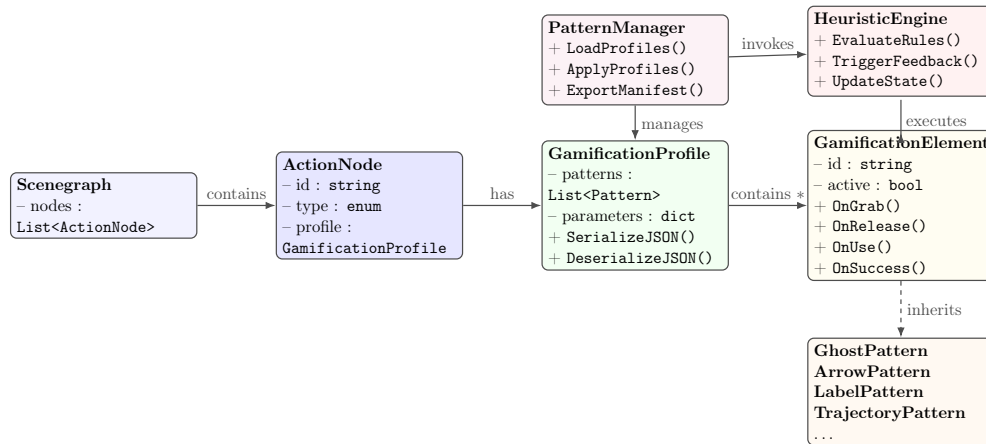


Figure 3.5: Data model of the Gamification Subsystem showing relationships between *Scenegraph*, *ActionNode*, *GamificationProfile*, and associated pattern and heuristic classes.

3.5.3 Core Layers

Scenegraph Layer (Authoring-time). The Scenegraph Layer serves as the structural foundation of the authoring environment. It represents the XR training procedure as a directed acyclic graph (DAG) of interconnected **Action Nodes**, each describing a discrete learner task such as *Insert*, *Use*, *Remove*. Each node encapsulates three primary data groups:

- (i) references to scene objects (Interactable, Target, or Path)
- (ii) behavioral metadata defining task parameters, success conditions, and transition triggers and
- (iii) an embedded Gamification Profile that stores the configuration of attached guidance and motivational elements.

From a technical perspective, nodes are implemented as serialized Scriptable Objects, ensuring persistence, reusability, and independence from specific Unity scenes. This design allows each Action Node to function as a self-contained data unit—storing both procedural logic and its associated feedback mechanisms within a single asset. Connections between nodes define the procedural order, conditional branching, and dependency hierarchy of the training scenario. The authoring interface visualizes these relationships interactively, enabling authors to create, connect, and rearrange tasks through a drag-and-drop workflow.

Each Action Node includes input and output ports that manage logical dependencies between actions, allowing branching or parallel task structures. Node connections are evaluated recursively at runtime to ensure proper activation and sequencing. Once authoring is complete, the scenegraph is exported as a structured JSON manifest, preserving all object references, parameters, and gamification settings. This manifest serves as the blueprint for runtime reconstruction, ensuring that the authored logic and its feedback mappings are faithfully reproduced during execution.

Gamification Layer (Authoring-time). The Gamification Layer operates as a dedicated subsystem within the Scenegraph, responsible for defining, configuring, and persisting all feedback-related behaviors. It is accessible directly from each **Action Node** through the **Gamification Panel**, a custom Unity editor window that allows authors to visually manage visualization patterns.

Each gamification element is implemented as a modular C# component that inherits from a common base interface, ensuring consistency and reusability across feedback types. This inheritance structure provides a unified interface for key life-cycle events, such as `OnGrab()`, `OnRelease()`, `OnUse()`, and `OnSuccess()`, ensuring that all feedback patterns follow consistent initialization, activation, and completion logic across different action types. When a new pattern is added through the panel, it is automatically instantiated as a serialized sub-object of the node's *GamificationProfile*, following Unity's `ScriptableObject` serialization model.

The Gamification Profile serves as a lightweight, structured data container that stores the configuration of all active feedback elements within a node. Editable parameters—such as color, animation speed, visibility rules, or material references—are exposed through Unity's serialized property system, allowing real-time adjustment without coding.

At the authoring level, this layer removes the need for manual prefab wiring or complex scripting normally required to connect instructional cues with interaction

logic. Instead, feedback definition becomes a visual, data-driven process integrated into the same workflow as procedural authoring. This design ensures that gamified feedback is treated not as an optional overlay but as a first-class, parameterized component of the Scenegraph.

Runtime Layer (Execution-time). At runtime, the serialized scenegraph and its embedded Gamification Profiles are parsed by a central **Pattern Manager**, which reconstructs the procedural flow and dynamically instantiates the corresponding interaction and feedback components. The manager acts as the runtime orchestrator, binding each Action Node to its scene objects and subscribing to key interaction events that drive the activation of gamification elements in real time.

Each feedback component follows a consistent lifecycle model inherited from the authoring layer. When an interaction event occurs, the Pattern Manager executes the corresponding lifecycle callbacks, activating, updating, or deactivating the relevant visual cues. For example, a Ghost visualization appears when a learner grabs an interactable object and gradually fades once it is correctly aligned, a Glyph or an Arrow dynamically points to off-screen or distant targets; and a Label provides contextual instructions or success confirmation when an action is completed. More complex elements, such as the Trajectory Line, continuously update as the user manipulates a tool or follows a predefined path, providing spatial feedback for actions like cutting, drawing, or guiding instruments.

Each instantiated pattern runs as a managed coroutine, controlling its visibility, timing, and animation without introducing performance overhead. Runtime updates are handled asynchronously, allowing multiple feedback components to operate concurrently while maintaining stable frame rates. Because gamification elements are data-driven rather than scene-specific, they can be loaded, modified, or replaced dynamically without affecting procedural integrity.

Runtime Rule Binding (Simplified View). At runtime, each Action Node emits lifecycle events (*OnStart*, *OnGrab*, *OnAlign*, *OnSuccess*, *OnUpdate*). The *Pattern Manager* listens for these events and applies a small set of declarative rules defined in the node’s *GamificationProfile*. Each rule specifies when and how a visual or motivational pattern (*e.g.*, *Ghost*, *Arrow*, *Label*) should respond to learner actions.

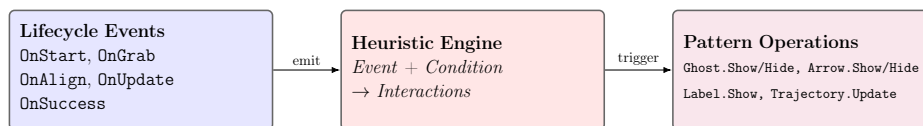


Figure 3.6: Runtime rule binding as an Event–Condition–Interaction (ECA) pipeline: node events feed the Heuristic Engine, which evaluates conditions and triggers concrete pattern operations.

Concept. A rule can be read as:

When <event> happens and <condition> holds, perform <pattern operations>.

This minimal schema—event, condition, and actions—captures most XR training behaviors without author scripting.

Example Rules for an *Insert*.

- **Start guidance:** on `OnStart`, show a **Ghost** at the target.
- **Highlight on grab:** on `OnGrab`, switch the Ghost to overlay mode.
- **Fade on alignment:** on `OnAlign`, hide the Ghost and show a success **Label**.
- **Off-screen pointing:** on `OnUpdate`, if the target leaves the field of view, show an **Arrow**; otherwise hide it.
- **Completion:** on `OnSuccess`, confirm success with a short **Label**.

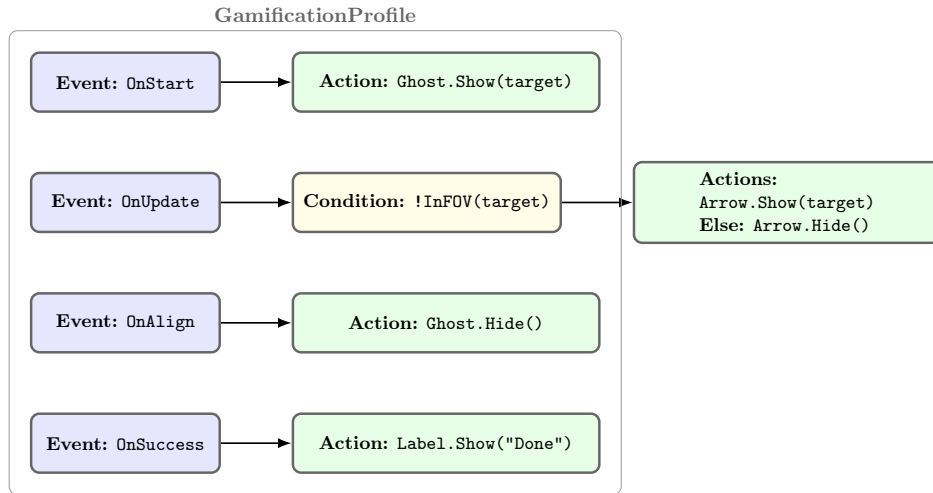


Figure 3.7: Visual representation of a declarative Gamification Profile for an *Insert*. Each rule follows an Event–Condition–Action schema linking interaction events (`OnStart`, `OnUpdate`, etc.) to pattern operations (e.g., `Ghost.Show()`, `Arrow.Show()`, `Label.Show()`).

Illustrative Manifest Snippet.

3.6 Pattern Library

This section describes the pattern library underlying the programmable-reality framework. It is divided into two complementary domains: **Visualization Patterns**—which define how feedback and guidance appear in 3D space—and **Interaction Patterns**, which formalize learner actions and procedural logic in the scenegraph. Together, they establish a unified vocabulary that bridges visual design and procedural authoring.

The visualization patterns correspond to the core gamification elements used throughout the framework. These include perceptual and motivational cues such as **arrows**, which direct the learner’s attention toward relevant objects or locations; **ghosts**, which provide semi-transparent holographic previews illustrating the correct position or motion of an object; and **panels** or **labels**, which deliver contextual information or textual feedback anchored in 3D space. Additional patterns include **trajectories**, which visualize ideal motion paths or tool movements; **virtual mirrors**, which allow learners to observe and adjust their own posture or performance; and **progress indicators**, which convey completion or performance feedback to sustain motivation. Together, these visualization structures transform

abstract feedback into spatially meaningful guidance that integrates seamlessly into the training environment.

Figure 3.8 illustrates the complete set of visualization and gamification elements implemented in the framework. It presents the six core feedback types—arrows, ghosts, panels, trajectories, mirrors, and progress indicators—highlighting how each serves a distinct perceptual or motivational function while maintaining visual coherence across training scenarios.

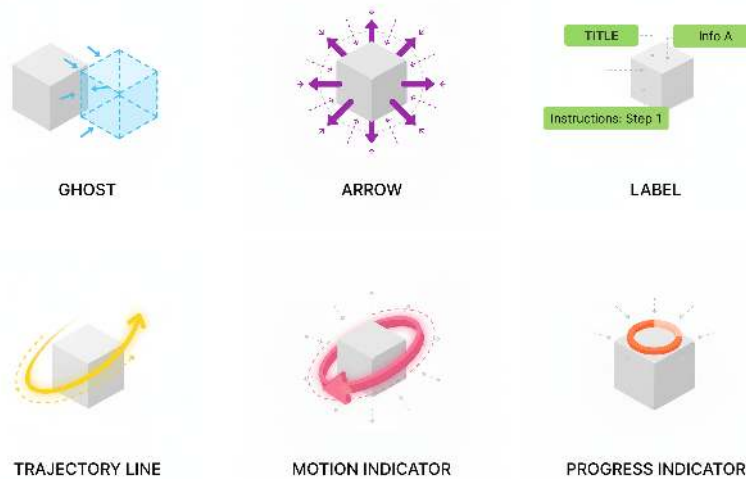


Figure 3.8: Overview of visualization patterns used in the framework.

Each visualization pattern can be parameterized and reused across multiple interaction types, ensuring consistency and scalability. For example, an *Insert* or *Use* action may automatically trigger a *Ghost* preview combined with an *Arrow* cue and contextual *Label*, forming a cohesive guidance sequence. In this way, gamification elements are treated not as decorative additions but as reusable, pedagogically informed components that structure feedback, direct attention, and sustain motivation within the programmable-reality framework.

Figure 3.9 further illustrates how these visualization patterns are mapped to specific interaction types within the authoring framework. It shows the default correspondence between procedural actions (e.g., *Insert*, *Use*, *Activate*) and the visualization cues that accompany them (e.g., *Ghost*, *Arrow*, *Label*). This mapping represents the declarative logic that underpins the gamification subsystem, ensuring consistency and pedagogical coherence across training scenarios.

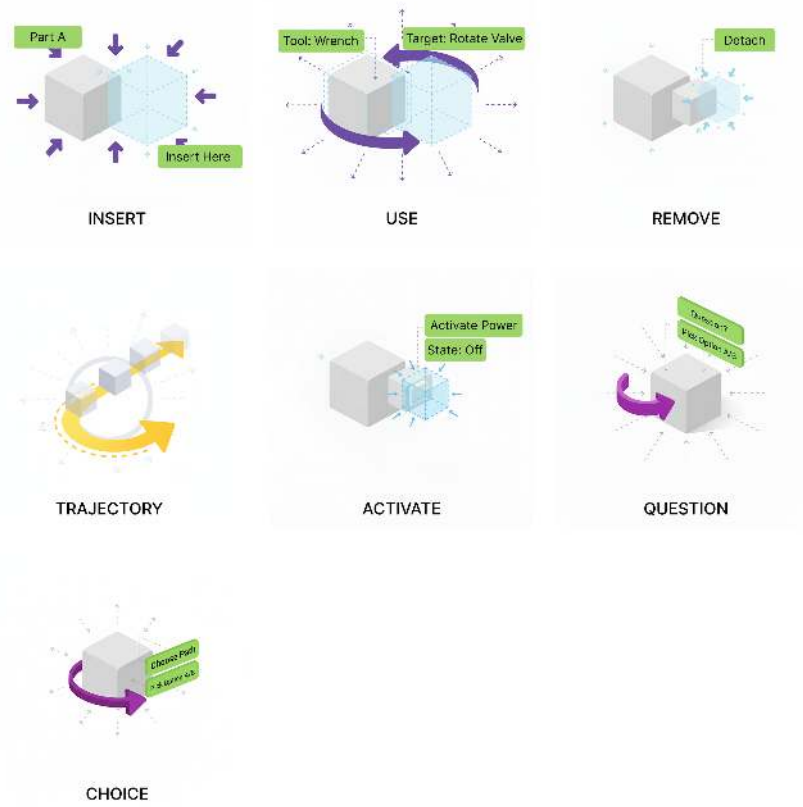


Figure 3.9: Mapping between visualization patterns and interaction patterns.

3.6.1 Our Interaction Patterns

Interaction patterns define the procedural building blocks of the programmable-reality framework. Each pattern encapsulates a specific learner action, expressed through standardized fields such as object bindings, conditions, and feedback triggers. This structure allows authors to assemble complex training sequences from modular, reusable actions without scripting.

The framework currently defines seven core **interaction patterns** that represent recurring learner actions in XR training scenarios. These include **Insert**, **Remove**, and **Use**, which capture fundamental procedural operations such as placing, detaching, or manipulating tools and components. The framework also introduces four extended patterns: **Activate**, used to trigger devices or mechanisms; **Trajectory**, which guides a tool or object along a continuous spatial path; **Question**, which presents in-situ cognitive checks or assessment steps; and **Choice**, which enables branching interactions or adaptive procedural outcomes. Together, these interaction patterns provide a reusable vocabulary for representing task logic and learner actions declaratively within the programmable-reality framework.

Each pattern exposes a consistent lifecycle, allowing the runtime to handle user actions, feedback synchronization, and success conditions deterministically. Extended implementation details and Unity-based setup tutorials for each Interaction Pattern are provided in Appendix C.

3.6.2 Our Visualization Patterns

The framework’s gamification subsystem is built around a structured library of reusable **interaction and visualization patterns**. These patterns bring the prin-

ciples of guidance and motivation directly into the authoring process. Each pattern supports a specific instructional goal—such as directing attention, illustrating motion, confirming actions, or signaling system state—while remaining modular and adaptable to diverse contexts. In practice, these patterns function as **templates** that can be instantiated, adjusted, and reused across multiple training scenarios, ensuring flexibility, scalability, and a significant reduction in development time compared to manual implementation.

Beyond the patterns derived from prior work, several custom gamification elements were designed to address specific authoring needs identified during XR training development. These include motion indicators for dynamic guidance, progress indicators for continuous task monitoring, and toggle cues for binary interaction feedback. Together, these additions extend the expressive capacity of the framework, providing authors with a comprehensive palette of guidance and motivational tools that can be seamlessly integrated into any procedural action. The Motion and Progress Indicators were introduced to cover two recurrent authoring needs observed across scenarios: communicating *how* to move (beyond *where* to place) and making continuous progress legible without bespoke scripts, respectively.

The library draws conceptual inspiration from the situated visualization taxonomy proposed by Lee et al. (2023) and extends it with interaction-oriented mechanisms from Zikas et al. (2023), thereby linking visualization design with procedural task modeling in XR training. All patterns follow a unified descriptive schema composed of four attributes:

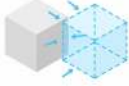
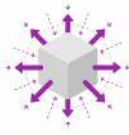
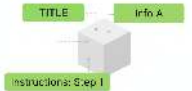
Visualization Pattern	Image	Intent / Context / Description
Ghost		Intent: Spatial guidance for alignment/assembly. Context: Insertion/assembly. Description: Semi-transparent preview of the target position that fades upon correct alignment.
Arrow (Glyph)		Intent: Direct attention to off-screen/distant targets. Context: When target exits the field of view. Description: Dynamic arrow orients to the target until visible.
Label		Intent: Provide contextual/instructional feedback. Context: On task start or completion. Description: Floating text prompts, hints, or confirmations.
Trajectory Line		Intent: Visualize controlled motion or paths. Context: Precise movement or assembly. Description: Dynamic curve updates as the learner moves the object.
Motion Indicator (new)		Intent: Illustrate expected movement (how to move). Context: Rotational/sweeping gestures (screwing, stirring). Description: Animated arrows/arcs convey motion; complements Ghosts (where to place).
Progress Indicator (new)		Intent: Visualize progress/completion. Context: Time-based or continuous actions (cutting, drilling). Description: Circular/linear bar fills in real time (0–100%).

Table 3.2: Stylized dictionary of implemented visualization and interaction patterns with representative images and concise descriptions, including newly introduced procedural and motivational cues.

To demonstrate how these visualization patterns are systematically embedded within the authoring framework, a mapping was established between each Interaction Pattern type and its corresponding default set of feedback elements. These pairings were designed not only for functional consistency but also for pedagogical coherence: each mapping reflects how learners perceive, act, and receive reinforcement during training. For example, spatially demanding actions such as Insert or Use require explicit perceptual scaffolds (Ghosts, Arrows) to reduce spatial uncertainty and direct attention, while cognitively demanding actions like Question or Choice emphasize motivational and reflective feedback (Labels) to sustain engagement and comprehension. Grounded in principles of guided discovery learning and cognitive load theory, these default mappings operationalize instructional intent as reusable design logic, ensuring that every procedural step provides the right amount of guidance and feedback at the right moment.

Table 3.3 summarizes these default mappings, illustrating how specific gamification elements are attached to core user actions to create a cohesive, immersive learning experience.

These mappings represent the **default configurations** automatically assigned to each Interaction Pattern upon creation. While they provide meaningful scaffolding out of the box, authors retain full control through the Gamification Panel. Any pattern can be **added, removed, or modified**, allowing developers to adapt guidance and feedback intensity according to specific training needs.

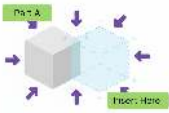
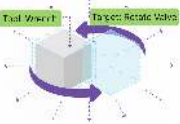
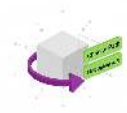
Interaction Pattern	Visualization Pattern	Example Illustration	Default Feedback Behavior
Insert	Ghost, Label, Arrow		A Ghost preview guides object placement; once grabbed, it shifts to the target. Labels provide contextual instructions, and an Arrow directs attention when elements move out of view.
Use	Ghost, Label, Arrow		Ghosts visualize both tool and target; Labels describe intended use, and an Arrow orients to off-screen items.
Remove	Ghost, Label		Ghost highlights the removable object and fades on successful detachment; Label confirms task completion.
Activate (new)	Ghost, Label		Ghost highlights the activatable zone; Label conveys purpose/state and confirms activation.
Trajectory (new)	Trajectory Line		Trajectory Line visualizes the ideal motion path and updates dynamically with user movement.
Question (new)	Arrow, Label		Arrow points to the question interface if out of view; Labels display contextual prompts above interactive elements.
Choice (new)	Arrow, Label		Arrow directs attention toward the available decision interfaces or selectable objects when out of view; Labels present contextual options and confirm the learner's selected path.

Table 3.3: Mapping between **Interaction Patterns** (procedural actions) and **Visualization Patterns** (feedback cues). For thumbnails and references to the primary descriptions of each pattern, see Appendix A.

3.7 System-Level and Algorithmic Validation

This section validates the framework as a **systems contribution** (Fatahalian, 2020), prioritizing architectural soundness, reproducibility, and algorithmic coherence over human-subject performance. In the spirit of pattern-based design work (Gamma et al., 1994), we evaluate how effectively the system unifies **Interaction Patterns** (procedural logic) and **Visualization Patterns** (motivational and visual feedback) into a single, declarative, and reproducible authoring model.

While Zikas et al. (2023) introduced reusable Interaction Patterns for procedural logic, this thesis extends that model by introducing Visualization Patterns that are structurally bound to each Action Node through shared lifecycle events. Together, they form a dual-layer pattern model that unifies interaction and motivation in a single declarative pipeline. Where the prior framework still required short scripts and manual prefab wiring for feedback, the extended system eliminates author code for gamification entirely. All parameters are specified declaratively, exported in the JSON manifest, and reconstructed by the Pattern Manager—constituting a **no-code** pipeline for feedback authoring.

Evaluation Strategy (Systems-Oriented). This thesis adopts a systems-style evaluation. In line with design-pattern and graphics systems work, we prioritize evidence of architectural soundness, complexity reduction, and reproducible behavior over end-user studies. Accordingly, we report code/complexity savings, authoring-time measures, and deterministic lifecycle execution as primary indicators of contribution. Summative user studies of authoring UX and learning impact are scoped as future work to build upon these systems results.

3.7.1 Validation Findings

Validation produced a set of key system-level observations demonstrating the framework’s structural integrity and operational consistency across both authoring and runtime layers:

Observation i: Authoring efficiency. The subsystem eliminated the need for manual coding by enabling declarative, no-code configuration of both procedural and feedback logic. Average authoring time per action decreased from approximately one day to one or two minutes, confirming substantial workflow acceleration.

Observation ii: Architectural reusability. Both Interaction and Visualization Patterns were parameterized as modular components, allowing them to be recombined, shared, and extended across training scenarios without rewriting logic. This composability confirmed that gamification behaviors function as reusable design primitives within the scenegraph model.

Observation iii: Behavioral determinism. Runtime analysis showed consistent synchronization between procedural and feedback layers. All Visualization Patterns executed lifecycle events deterministically (activation, update, completion), ensuring that feedback timing aligned precisely with learner actions.

Observation iv: Design–runtime coherence. Authored configurations, serialized as JSON manifests, were reconstructed faithfully at runtime with no deviation in procedural flow or feedback presentation. This verified the coherence of the declarative data model and the robustness of the pattern-based serialization pipeline.

This validation focuses on the framework as a software system rather than a user-facing product. The goal is to ensure that architectural mechanisms, operate as designed. Validation therefore evaluates the internal behavior and reproducibility of the framework itself, establishing its readiness for subsequent user-centered studies.

Research Question Traceability. The methodological design of this work directly operationalizes the three research questions introduced in Chapter 1. Each question is mapped to the corresponding methodological elements and validation evidence presented in this chapter.

- **RQ1:** Addressed through the formalization of feedback, guidance, and motivational logic as reusable, lifecycle-bound **Visualization Patterns** embedded within each **Action Node**. This is detailed in *System Architecture and Implementation* (Section 3.5) and the *Pattern Library* (Section 3.6).
- **RQ2:** Examined by unifying procedural **Interaction Patterns** and **Visualization Patterns** into a single programmable framework with synchronized lifecycles and default mappings. See *Pattern Library—Interaction Patterns* (Section 3.6.1) and *Pattern Library—Visualization Patterns* (Section 3.6.2), together with the lifecycle bindings described in *System Architecture and Implementation* (Section 3.5).
- **RQ3:** Supported by system-level validation showing reductions in authoring time and code volume, improved scalability, and deterministic runtime behaviour. See *System-Level and Algorithmic Validation* (Section 3.7), *Authoring Workflow Efficiency* (Section 3.7.2), *Representative System Comparison* (Section 3.7.3), and *Quantitative System Metrics* (Section 3.7.4), as well as Table 3.4 and Figure 3.16.

3.7.2 Authoring Workflow Efficiency

One of the key goals of this work was to reduce authoring complexity and improve development efficiency. Traditional Unity-based workflows rely on repetitive coding and manual setup for each gamified behavior. In contrast, the integrated **Gamification Panel** provides a visual, no-code interface where authors can add, remove, or modify feedback elements directly from each Action Node (Figure 3.10). The panel supports dynamic configuration of visual properties, allowing authors to instantly adjust color, opacity, material, and animation themes for feedback patterns. Changes are reflected in real time in the editor viewport, enabling rapid iteration and consistent visual feedback across training scenarios without requiring code modifications.



(a) Ghost pattern with green color theme. (b) Ghost pattern with pink color theme.

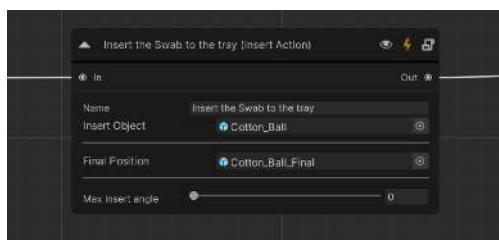
Figure 3.10: Dynamic configuration of the Ghost pattern through the Gamification Panel.

This immediate, code-free parameterization exemplifies the thesis’s shift from low-code scripts to a fully no-code, data-driven configuration of feedback.

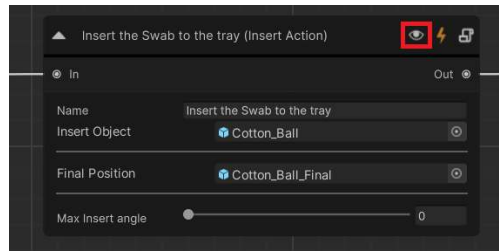
Representative Authoring Scenario. To illustrate the simplified workflow, consider the setup of three elements—**Ghost**, **Arrow**, and **Label**—each used in common XR training actions.

1. Ghost Pattern (Insert). Within the Scenegrph Editor, the author selects the corresponding Action Node (Figure 3.11a) and opens its Gamification Panel (Figure 3.11b). By default, the node includes both a Ghost and a Label pattern. The author can visually configure each pattern, for instance, enabling *Show on Interactable* to make a cotton ball flash at the start of the action and *Show on Target* to project a holographic preview over the correct placement area. If the Label element is unnecessary, it can be removed instantly using the *Remove Element* option (Figure 3.11e). Once these settings are saved, the Scenegrph automatically serializes the configuration into the node’s *GamificationProfile*. At runtime,

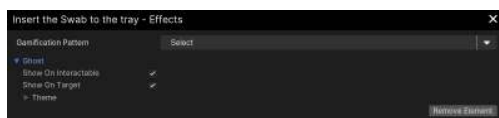
the system reconstructs these parameters, displaying the Ghost overlay and visual feedback automatically, without any manual scripting or prefab setup, as demonstrated across Figures 3.11a–d.



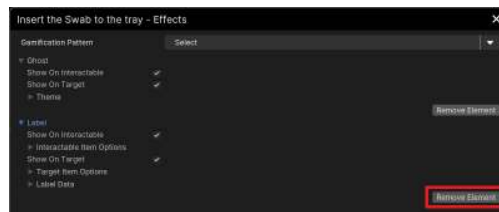
(a) Action Node setup.



(b) Opening the panel.



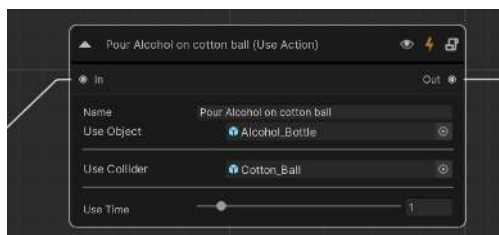
(c) Ghost/Label configuration.



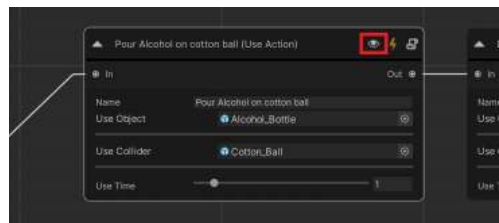
(d) Ghost visibility setup.

Figure 3.11: Representative authoring workflow for configuring the Ghost pattern.

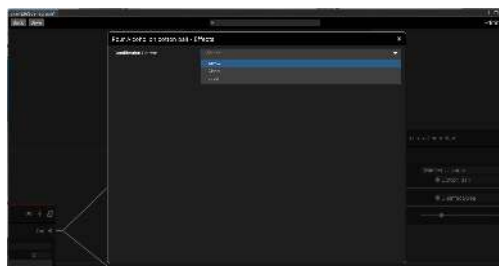
2. Arrow Pattern (Use). For a *Use*, such as applying disinfectant with a bottle and cotton ball, the Arrow pattern provides directional guidance toward the interactable object. From the Scenegraph Editor, the author selects the corresponding Action Node (Figure 3.12a) and opens the Gamification Panel via the eye icon (Figure 3.12b). From the dropdown list, the *Arrow Pattern* is selected, and the option *Show on Interactable* is enabled (Figure 3.12c). This automatically spawns an arrow pointing toward the alcohol bottle, helping users locate it in 3D space. Authors can override the arrow asset if a custom visual is required or adjust visibility parameters (Figure 3.12d). After saving the configuration, the Scenegraph serializes it, and at runtime, the arrow is automatically positioned and activated when the action begins—again, without coding or prefab adjustments.



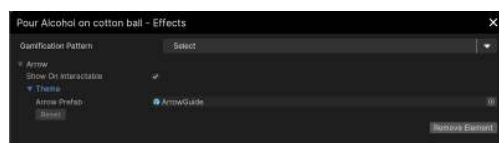
(a) Arrow-enabled Action Node.



(b) Opening the panel.



(c) Arrow configuration.

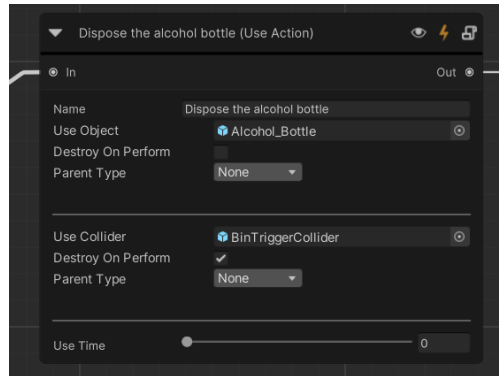


(d) Arrow visibility options.

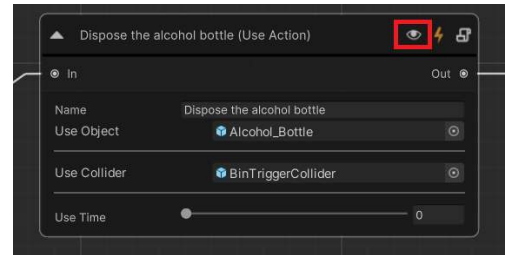
Figure 3.12: Authoring workflow for configuring the Arrow gamification element.

3. Label Pattern (Use or Insert). The Label pattern displays contextual text above

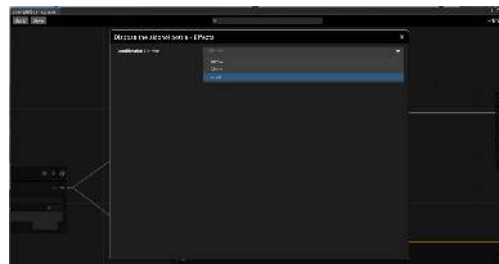
relevant objects or targets, such as “Pick up bottle” or “Dispose in bin.” In a *Use*, the author opens the Gamification Panel (Figure 3.13b) and selects the *Label Pattern* (Figure 3.13c). Both *Show on Interactable* and *Show on Target* options can be enabled, displaying text above the source object and destination collider respectively. Optional *Label Trajectories* can be activated to draw connecting lines between labels and their corresponding objects (Figure 3.13d). Once saved, these parameters are serialized and reconstructed automatically at runtime, producing the contextual overlays shown in Figure 3.13e.



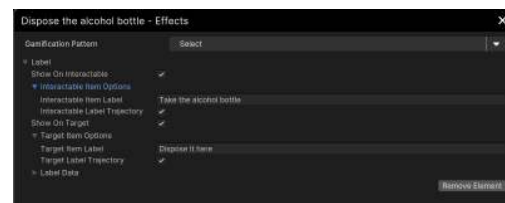
(a) Label Action Node.



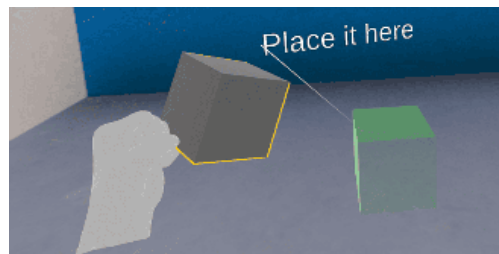
(b) Opening the panel.



(c) Label configuration.



(d) Label trajectories.



(e) Labels applied in context.

Figure 3.13: Authoring workflow for configuring the Label gamification element.

Collectively, Figures 3.11–3.13 demonstrate how the Gamification Subsystem transforms feedback creation from a code-centric to a data-driven and visually configurable process. Authors can combine multiple feedback elements directly within the Scenegraph without writing code, ensuring consistent behavior and faster iteration.

3.7.3 Representative System Comparison

To demonstrate system-level efficiency, we compare three generations of XR authoring workflows: (a–b) manual Unity scripting (Figure 3.14), (c) the scenegraph/action-pattern framework of Zikas et al. (2023) (Figure 3.15, and (d) the extended pattern-driven model introduced in this thesis, which integrates Visualization Patterns as first-class, lifecycle-bound components (Figure 3.16).

```
1 public class CraneMovement : MonoBehaviour
2 {
3     [SerializeField]
4     float moveSpeed = 3;
5
6     [SerializeField]
7     float chainMoveSpeed = 3;
8
9     [SerializeField]
10    float chainRotationSpeed = 3;
11
12    [SerializeField]
13    GameObject chainAnchor;
14
15    // Update is called once per frame
16    void Update()
17    {
18        // Move Forward/Backward
19        transform.Translate(Vector3.forward * Time.deltaTime * Input.GetAxis(
20            "Vertical") * moveSpeed);
21
22        // Move Left/Right
23        transform.Translate(Vector3.right * Time.deltaTime * Input.GetAxis(
24            "Horizontal") * moveSpeed);
25
26        // Crane Up/Down
27        if (Input.GetKey("left shift") && chainAnchor.transform.localPosition.y <=
28            8.0f)
29        {
30            chainAnchor.transform.Translate(Vector3.up * chainMoveSpeed * Time.
31                deltaTime);
32        }
33        else if (Input.GetKey("space") && chainAnchor.transform.localPosition.y >=
34            5.0f)
35        {
36            chainAnchor.transform.Translate(Vector3.down * chainMoveSpeed * Time.
37                deltaTime);
38        }
39
40        // Crane Rotates
41        if (Input.GetKey("d")) // Clockwise
42        {
43            chainAnchor.transform.Rotate(0, 90.0f * Time.deltaTime *
44                chainRotationSpeed, Space.Self);
45        }
46        else if (Input.GetKey("a")) // CounterClockwise
47        {
48            chainAnchor.transform.Rotate(0, 90.0f * Time.deltaTime *
49                chainRotationSpeed, Space.Self);
50        }
51    }
52 }
```

(a) Manual Unity Insert Interaction(part 1 of 2).

```
1 public class Crane : MonoBehaviour
2 {
3     [SerializeField]
4     float moveSpeed = 3;
5
6     [SerializeField]
7     float chainMoveSpeed = 3;
8
9     [SerializeField]
10    float chainRotationSpeed = 3;
11
12    [SerializeField]
13    GameObject chainAnchor;
14
15    // Update is called once per frame
16    void Update()
17    {
18        // Move Forward/Backward
19        transform.Translate(Vector3.forward * Time.deltaTime * Input.GetAxis(
20            "Vertical") * moveSpeed);
21
22        // Move Left/Right
23        transform.Translate(Vector3.right * Time.deltaTime * Input.GetAxis(
24            "Horizontal") * moveSpeed);
25
26        // Crane Up/Down
27        if (Input.GetKey("left shift") && chainAnchor.transform.localPosition.y <=
28            8.0f)
29        {
30            chainAnchor.transform.Translate(Vector3.up * chainMoveSpeed * Time.
31                deltaTime);
32        }
33        else if (Input.GetKey("space") && chainAnchor.transform.localPosition.y >=
34            5.0f)
35        {
36            chainAnchor.transform.Translate(Vector3.down * chainMoveSpeed * Time.
37                deltaTime);
38        }
39
40        // Crane Rotates
41        if (Input.GetKey("d")) // Clockwise
42        {
43            chainAnchor.transform.Rotate(0, 90.0f * Time.deltaTime *
44                chainRotationSpeed, Space.Self);
45        }
46        else if (Input.GetKey("a")) // CounterClockwise
47        {
48            chainAnchor.transform.Rotate(0, 90.0f * Time.deltaTime *
49                chainRotationSpeed, Space.Self);
50        }
51    }
52 }
```

(b) Manual Unity Insert Interaction(part 2 of 2).

Figure 3.14: Manual Unity implementation of the **Insert Interaction** showing the complete C# scripting workflow (≈ 150 lines of code).

```
1 public class PolyethyleneIrialAction : InsertAction
2 {
3     public override void Initialize()
4     {
5         SetInsertPrefab("Lesson7/Stage2/Action0/Polyethylene",
6             "Lesson7/Stage2/Action0/PolyethyleneFinal");
7         SetHoloObject("Lesson7/Stage2/Action0/Hologram/HologramL752A0");
8
9         base.Initialize();
10    }
11 }
```

Figure 3.15: Scenegraph-based implementation of the **Insert Interaction** in the baseline framework by Zikas et al. (2023), configured through a compact 5-line JSON specification.

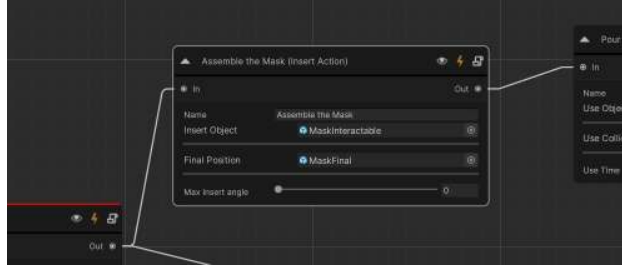


Figure 3.16: Declarative, no-code configuration of the **Insert Interaction** in the extended, pattern-driven framework developed in this thesis.

While Zikas et al. (2023) established reusable Interaction Patterns for procedural logic, feedback and motivation remained external to the system, often implemented as ad-hoc prefabs or manually linked scripts. The extended framework addresses this limitation by embedding Visualization Patterns, modular components such as *Ghost*, *Arrow*, and *Label*, directly into the same architectural layer as the Action Nodes. Each gamification component follows a unified lifecycle interface, mirroring the event structure of its parent Interaction Pattern. This design ensures that feedback and procedural logic progress in strict temporal alignment: when an action is triggered or completed, its corresponding visual and motivational responses activate and terminate deterministically.

At the authoring level, this integration is realized through the Gamification Panel, which allows configuration of feedback elements as serialized sub-objects within each Action Node’s GamificationProfile. All parameters, such as visibility rules, animation duration, asset references, or activation thresholds, are stored alongside the procedural meta-data and exported within the same JSON manifest. At runtime, the Pattern Manager parses this manifest, reconstructing both the procedural graph and its associated gamification layer as a single, interdependent data structure. This eliminates the need for separate prefabs or external logic, guaranteeing that authored intentions are faithfully reconstructed during execution.

Architecturally, this binding transforms the framework from a procedural scripting tool into a declarative co-authoring environment, where guidance and motivation are treated as first-class entities rather than supplementary effects. Pedagogically, it enables a consistent mapping between learner actions, visual feedback, and motivational cues across different training scenarios. Because all Interaction and Visualization Patterns share the same data model, they can be reused, extended, or combined freely, supporting scalable authoring while maintaining behavioral determinism and design coherence across XR platforms.

3.7.4 Quantitative System Metrics

This subsection defines and reports the **system-level metrics** used to evaluate the technical performance of the proposed framework. The authoring-time, code-volume, and runtime-cost metrics reported in Table 3.4 were derived from direct implementation comparisons across three authoring workflows. The **Manual Unity** baseline was reconstructed by implementing a standard *Insert Interaction* (object placement with success feedback) using conventional Unity scripting practices, resulting in approximately 150 lines of C# code and multiple prefab linkages. The **Scenegraph/Interaction Patterns** baseline corresponds to the publicly available SDK described by Zikas et al. (2023), which implements the same action as a 5-line configuration using serialized nodes. The **Extended Framework** represents the new pattern-driven system introduced in this thesis, where equivalent functionality is defined declaratively through the Scenegraph and integrated *Gamification Panel* without any user-authored code. All three implementations were verified to produce equivalent runtime behaviour, ensuring a fair comparison of authoring effort, code complexity, and computational performance.

Across these baselines, the extended framework achieved a substantial improvement in authoring efficiency and maintainability. Average authoring time per action decreased from approximately one day to one or two minutes, and code volume was reduced from

about 150 lines of C# to a fully declarative, zero-code configuration. Runtime profiling confirmed that the heuristic engine introduced less than 2% frame-rate deviation relative to the baseline, indicating negligible computational overhead. Together, these results validate that the proposed framework delivers a significantly **faster, easier, and computationally efficient** authoring process while maintaining deterministic runtime behaviour and full scalability through JSON-based data serialization.

Validation Criterion	Manual Unity	Scenegraph / Interaction Patterns (Zikas et al., 2023)	This Thesis (Extended)
Authoring effort (per Interaction)	~1 day	~10–15 min	~1–2 min
Code volume (per Interaction)	~150 lines (C#)	~5 lines	0 (declarative)
Feedback integration	Ad-hoc prefabs (manual setup)	Separate or manual per step	Built-in
Runtime determinism	Manual event wiring	Deterministic procedural flow	Deterministic procedural flow
Configuration interface	None	Visual editor for actions	Visual editor + Gamification Panel
Scalability / portability	Scene-dependent logic	Serialized nodes (partial)	JSON manifest (full pipeline)

Table 3.4: System-level validation summary across three authoring stages: manual Unity implementation, the scenegraph/action-pattern baseline (Zikas et al., 2023), and the extended pattern-driven framework developed in this thesis.

Chapter 4

Conclusions and Future Work

“Simple things should be simple, complex things should be possible.”

— Alan Kay, *Turing Award Lecture* (2004)

This thesis introduced the design, implementation, and evaluation of a **pattern-driven gamification framework for programmable reality**, an integrated approach that unifies procedural interaction and spatial visualization patterns into a coherent authoring methodology for XR training. Built upon the scenegraph-based framework of Zikas et al. (2023), the work reconceptualizes feedback, guidance, and motivation as **first-class, reusable, data-defined components** rather than ad hoc scripts or prefabs.

We have demonstrated that **formalizing gamification as declarative, pattern-based components** *significantly lowers the barrier* to creating effective and engaging XR training. In doing so, the framework moves the field **from craft-based scripting to systematic, pattern-based authoring**: authors configure interaction and feedback through no-code panels, the system serializes these decisions into portable manifests, and a runtime orchestrator executes them deterministically. This shift yields faster authoring, greater consistency, and clearer alignment between instructional intent and immersive behavior.

At the core of this framework lies the concept of **programmable reality**: immersive environments in which interaction and feedback behaviors are specified declaratively through pattern logic rather than manual scripting. Within this model, guidance and gamification are intrinsic to the learning design process, authored visually and executed through lifecycle-bound algorithms that ensure consistency, scalability, and pedagogical coherence.

The evaluation of this work followed a systems-oriented methodology, emphasizing the framework’s architectural integrity and practical efficiency rather than end-user perception. Instead of focusing on subjective usability or learning outcomes, the assessment targeted quantifiable indicators such as authoring-time reduction, code complexity, and deterministic execution behavior. These metrics capture the system’s contribution to reducing design overhead and improving reproducibility in XR authoring workflows. While this focus aligns with design-pattern and interactive-systems research traditions, future work will extend the evaluation to user-centered studies, examining the authoring experience and pedagogical effectiveness in real-world training contexts.

4.1 Summary of Contributions

The main contributions of this work can be summarized as follows:

- **Formalized gamification as reusable system components.** Feedback and guidance features were modeled as **Gamification Profiles**, data structures attached to **Action Nodes** within the scenegraph. This design enabled direct configuration of gamified elements such as Ghosts, Arrows, and Labels, ensuring consistent and scalable behavior across training scenarios.

- **Developed algorithmic heuristics for feedback management.** The subsystem introduced lifecycle-based heuristics that governed when and how feedback appeared, behaved, and disappeared. These rules automated the timing and logic of gamification, reducing manual intervention while preserving pedagogical intent.
- **Implemented visual authoring through the Gamification Panel.** A dedicated authoring interface allowed designers to configure all feedback patterns through dropdown menus and parameter fields, supporting no-code customization for users without programming expertise.
- **Validated improvements in authoring efficiency and reusability.** Quantitative evaluation (Section 3.7.4) demonstrated a substantial reduction in authoring time and code complexity compared to manual Unity workflows. Specifically, the extended framework reduced authoring effort from approximately **one day to one or two minutes per interaction**, and code volume from about **150 lines of C# to a fully declarative, zero-code configuration**. Because interaction and feedback behaviors are defined as **reusable patterns** rather than fixed scripts, they can be efficiently reapplied across multiple XR scenarios, enhancing both development speed and design consistency. These results confirm that the proposed pattern-driven approach significantly accelerates XR content production while maintaining deterministic runtime behavior and architectural scalability.

By embedding gamified feedback logic within the scenegraph itself, the framework supports reproducibility, scalability, and long-term maintainability in immersive learning environments.

4.2 Discussion and Implications

This work advances the state of XR authoring by bridging three domains that are typically treated separately: *procedural interaction*, *spatial visualization*, and *gamification*. By unifying these through a common pattern language, the framework closes the gap between instructional design intent and its technical realization in an XR environment.

From an engineering standpoint, the declarative Scenegraph architecture enables clean separation between authored logic and runtime execution. Action and Gamification Profiles act as portable, data-driven templates that can be instantiated dynamically across different XR contexts, supporting reusability and interoperability.

From a design perspective, the system operationalizes key pedagogical principles such as scaffolding, progression, and feedback consistency. Educators and developers can co-author scenarios through visual, pattern-based workflows, ensuring that guidance and motivation are not afterthoughts but intrinsic parts of the instructional structure.

In doing so, the framework democratized XR authoring: by removing the dependency on coding skills and exposing gamification and interaction logic through intuitive visual tools, it enabled a wider range of educators, trainers, and domain experts to participate directly in content creation. This accessibility transforms XR development from a specialized engineering task into a collaborative design process grounded in pedagogical intent.

4.3 Future Work

Future research should further extend the capabilities of the Gamification Subsystem toward adaptive, intelligent, and data-driven authoring workflows. One promising direction involves the development of adaptive and AI-assisted feedback mechanisms. By coupling the subsystem with real-time learner analytics or reinforcement learning models, future implementations could dynamically adjust the timing, intensity, or modality of feedback based on user performance and context. Such adaptivity would enable training scenarios to respond intelligently to learner behavior, maintaining engagement while optimizing cognitive load.

A second major avenue concerns the expansion of heuristic rules and validation mechanisms. At present, gamification logic is governed by deterministic lifecycle events. Intro-

ducing automated rule-checking and conflict-resolution systems could enhance the robustness of feedback orchestration, particularly in complex, multi-branch scenarios. These mechanisms would detect redundant or contradictory cues and apply context-sensitive prioritization of visual elements, ensuring coherence and reducing perceptual overload for learners.

In addition, future work should focus on extending the system through template-based authoring tools and domain-specific libraries. Expanding the catalog of default Gamification Profiles and providing reusable templates for domain-specific procedures—such as medicine, manufacturing, or maintenance—would further reduce authoring time while promoting standardization and pedagogical consistency across training domains.

Another essential direction is the empirical evaluation of the framework’s educational impact. Future studies should examine usability, workload, engagement, and learning outcomes using standardized measures such as the System Usability Scale (SUS) and NASA-TLX. Comparative analyses against baseline or non-gamified authoring workflows would clarify which pattern bundles yield the greatest pedagogical and motivational benefits for specific action types, contributing evidence-based insights to XR instructional design.

A further promising direction involves the development of AI-assisted authoring support systems. Instead of relying solely on manual configuration within the Gamification Panel, future iterations could integrate generative or predictive models that automatically suggest suitable Interaction Patterns, feedback parameters, or visual cues based on scene context and instructional intent. For example, a trained model could infer where a *Ghost* or *Arrow* should appear based on object proximity and task semantics, or propose optimal parameter values such as visibility thresholds and animation durations. In an immersive authoring environment, such a system could even operate directly within the XR editor, allowing designers to receive intelligent suggestions or automated adjustments in real time. This form of AI-assisted co-authoring would transform the framework from a declarative tool into an intelligent design partner, capable of streamlining workflows and enhancing pedagogical consistency across training scenarios.

Finally, a particularly significant research direction lies in exploring the use of visualization patterns as structured data for embodied AI systems. Because each Action Node encodes procedural logic, interaction semantics, and spatial feedback, the scene-graph architecture provides a rich source of structured data for autonomous agents to learn procedural reasoning, spatial understanding, and instructional sequencing. Future investigations could explore how reinforcement learning or generative models leverage these gamified structures to emulate expert-like guidance or autonomously generate new XR training content. This connection between human-centered authoring and machine learning opens the door to hybrid XR–AI frameworks in which visualization patterns serve both as pedagogical constructs and as training data for intelligent instructional agents.

Chapter 5

Bibliography

- [1] CreatXR. The virtuality spectrum — understanding ar, mr, vr and xr, 2022. URL <https://creatxr.com/the-virtuality-spectrum-understanding-ar-mr-vr-and-xr/>. Accessed: September 23, 2025.
- [2] Growth Engineering. Edgar dale’s cone of experience: A comprehensive guide. <https://www.growthengineering.co.uk/what-is-edgar-dales-cone-of-experience/>, 2023. Blog / educational resource.
- [3] Bongshin Lee, Bahador Saket, Nicolai Marquardt, and Barrett Ens. Design patterns for situated visualization in augmented reality. *IEEE Transactions on Visualization and Computer Graphics*, 2023. doi: 10.1109/TVCG.2023.3242231.
- [4] Robert M. Gagné. *The Conditions of Learning and Theory of Instruction*. Holt, Rinehart and Winston, New York, 4th edition, 1985.
- [5] Growth Engineering. Nine events of instruction. URL <https://www.growthengineering.co.uk/nine-events-of-instruction/>. Accessed: 22 October 2025.
- [6] Paul Zikas, Antonis Protopsaltis, Nick Lydatakis, Mike Kentros, Stratos Geronikolakis, Steve Kateros, Manos Kamarianakis, Giannis Evangelou, Achilleas Filipidis, Eleni Grigoriou, Dimitris Angelis, Michail Tamiolakis, Michael Dodis, George Kokiadis, John Petropoulos, Maria Pateraki, and George Papagiannakis. Mages 4.0: Accelerating the world’s transition to vr training and democratizing the authoring of the medical metaverse. *IEEE Computer Graphics and Applications*, 43(2):43–56, 2023. doi: 10.1109/MCG.2023.3242686.
- [7] Christopher Alexander, Sara Ishikawa, and Murray Silverstein. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, 1977.
- [8] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA, 1994. ISBN 978-0201633610.
- [9] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work? a literature review of empirical studies on gamification. In *47th Hawaii International Conference on System Sciences (HICSS)*, pages 3025–3034, 2014. doi: 10.1109/HICSS.2014.377.
- [10] Sebastian Deterding, Dan Dixon, Rilla Khaled, and Lennart Nacke. Gamification: Using game design elements in non-gaming contexts. *CHI ’11 Extended Abstracts on Human Factors in Computing Systems*, pages 2425–2428, 2011. ISSN 2156-3950. doi: 10.1145/1979742.1979575.
- [11] K. Suzuki, J. Zimmer, P. Fung, et al. Toward programmable reality: Modular xr design patterns for next-generation training systems. *Frontiers in Virtual Reality*,

- 6:1649785, 2025. URL <https://www.frontiersin.org/articles/10.3389/frvir.2025.1649785/full>. Accessed: October 4, 2025.
- [12] J. Radianti, C. Majchrzak, and T. Fromm. What is xr? towards a framework for augmented and virtual reality. *Computers & Education*, 183:104495, 2022. doi: 10.1016/j.compedu.2022.104495.
 - [13] Jonathan Pottle. The role of extended reality technology in healthcare education: Toward a research agenda. *Advances in Simulation*, 6:19, 2021. doi: 10.1186/s41077-021-00168-2.
 - [14] Paul Milgram and Fumio Kishino. A taxonomy of mixed reality visual displays. *IEICE Transactions on Information and Systems*, E77-D(12):1321–1329, 1994.
 - [15] Konstantina Kilteni, Raphaela Groten, and Mel Slater. The sense of embodiment in virtual reality. *Presence Teleoperators & Virtual Environments*, 21, 11 2012. doi: 10.1162/PRES_a_00124.
 - [16] I-Chun Hung, Lung-I Lin, Wei-Chieh Fang, and Nian-Shing Chen. Learning with the body: An embodiment-based learning strategy enhances performance of comprehending fundamental optics. *Interacting with Computers*, 26:360–371, 07 2014. doi: 10.1093/iwc/iwu011.
 - [17] Franck Ganier, Charlotte Hoareau, and Jacques Tisseau. Evaluation of procedural learning transfer from a virtual environment to a real situation: A case study on tank maintenance training. *Ergonomics*, 57, 03 2014. doi: 10.1080/00140139.2014.899628.
 - [18] Kevin Werbach and Dan Hunter. *For the Win: How Game Thinking Can Revolutionize Your Business*. Wharton Digital Press, 2012. ISBN 978-1613630235.
 - [19] Karl M. Kapp. *The Gamification of Learning and Instruction: Game-Based Methods and Strategies for Training and Education*. Pfeiffer, 2012. ISBN 978-1118096345.
 - [20] Airbus. Virtual procedure trainer (vpt). <https://aircraft.airbus.com/en/services/train/flight-training-solutions/virtual-procedure-trainer>, 2025. Accessed for thesis context.
 - [21] VRpilot. Virtual reality solutions for professional flight training: Vrflow. <https://vrpilot.aero>, 2025. Accessed for context in thesis.
 - [22] Edgar Dale. *Audio-Visual Methods in Teaching*. Dryden Press, New York, 1946.
 - [23] Rafael Molina-Carmona, Rubén Satorre-Cuerda, Inmaculada Coma, and José Ramón Hilera González. A model for gamification of training in augmented reality for the workplace. *Applied Sciences*, 10(1):274, 2020. doi: 10.3390/app10010274.
 - [24] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
 - [25] John Sweller, Paul Ayres, and Slava Kalyuga. *Cognitive Load Theory*. Springer, New York, 2011.
 - [26] Mina C. Johnson-Glenberg. Immersive vr and education: Embodied design principles that include gesture and hand controls. *Frontiers in Robotics and AI*, 5:81, 2018. ISSN 2296-9144. doi: 10.3389/frobt.2018.00081.
 - [27] Lev S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1978.
 - [28] Jean Lave and Etienne Wenger. *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press, Cambridge, 1991.
 - [29] Jan Borchers. *A Pattern Approach to Interaction Design*. Wiley, 2001.

- [30] Andy Dearden and Janet Finlay. Pattern languages in hci: A critical review. *Human-Computer Interaction*, 21(1):49–102, 2006. doi: 10.1207/s15327051hci2101_3.
- [31] Jenifer Tidwell. *Designing Interfaces: Patterns for Effective Interaction Design*. O’Reilly Media, 2nd edition, 2011. ISBN 978-1449379704.
- [32] Lucas Queiroz, Vanessa Silva, and Cláudio Farias. Evaluating cognitive load and presence in immersive virtual learning environments. *Virtual Reality*, 2023. ISSN 1434-9957. doi: 10.1007/s10055-023-00708-9.
- [33] Telmo Zarraonandia, Paloma Díaz, and Ignacio Aedo. Gamification in immersive environments: A literature review. *Multimodal Technologies and Interaction*, 3(4):67, 2019. doi: 10.3390/mti3040067.
- [34] Wolfgang Ketter, Markus Peters, and John Collins. A Multiagent Competitive Gaming Platform to Address Societal Challenges. *MIS Quarterly*, 40(2):447–460, 2016. ISSN 0276-7783. doi: 10.25300/MISQ/2016/40.2.09.
- [35] Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain*. David McKay Company, New York, 1956.
- [36] M. David Merrill. First principles of instruction. *Educational Technology Research and Development*, 50(3):43–59, 2002. doi: 10.1007/BF02505024.
- [37] Kayvon Fatahalian. What makes a (graphics) systems paper beautiful. 2020. URL <https://graphics.stanford.edu/~kayvonf/notes/systemspaper/>. Accessed October 2025.

Appendix A

Appendix

This appendix provides supplementary implementation details that support the methodology presented in Chapter 3. It documents the technical configuration, procedural setup, and runtime behavior of the reusable pattern components that constitute the programmable-reality framework.

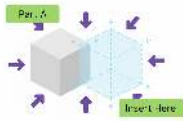
The appendix is organized into three main parts:

- **Appendix A.1 — Pattern Catalogs:** Tabulated references of all implemented *Interaction* and *Visualization Patterns*, including representative images, provenance sources, and descriptions of their purpose and integration within the framework.
- **Appendix A.2 — Pattern Setup Tutorials:** Step-by-step configuration guides for both *Visualization Patterns* (Ghost, Arrow, Label, Motion, Progress) and *Action Patterns* (Activate, Trajectory, Question, Choice), detailing their implementation in the Unity Scenegraph workflow.
- **Appendix A.3 — Algorithmic Heuristics and Runtime Manifest:** Formalization of the declarative rule system that governs runtime feedback logic, showing how pattern behaviors (e.g., visibility, alignment, success feedback) are dynamically evaluated through serialized heuristic rules.

Together, these sections provide a reproducible technical reference for implementing and extending the pattern-driven XR training framework. They complement the conceptual discussion of Chapter 3 by presenting concrete Unity-based implementation steps, serialized data structures, and runtime heuristics that operationalize the proposed approach.

A Pattern Catalog

A.1 Interaction Patterns (Procedural)

Interaction Pattern	Image	Reference (source / description)
Insert		Zikas et al. [6] (scenegraph SDK action pattern); used as procedural base in this thesis.

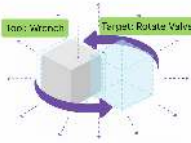
Interaction Pattern	Image	Reference (source / description)
Use		Zikas et al. [6] (SDK pattern); tool/target operation semantics.
Remove		Zikas et al. [6] (SDK pattern); detachment semantics.
Activate (new)		<i>This thesis</i> (new procedural action for trigger-based state changes; see Section 3.6.1).
Trajectory (new)		<i>This thesis</i> (new; continuous motion along a spline/path; see Section 3.6.1).
Question (new)		<i>This thesis</i> (new; in-situ knowledge checks integrated in scenegraph; see Section 3.6.1).
Choice (new)		<i>This thesis</i> (new; procedural branching interaction enabling decision-based scenario flow; see Section 3.6.1).

Table A.1: Interaction Pattern catalog with representative thumbnails and provenance references.

A.2 Visualization Patterns (Feedback)

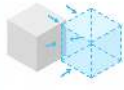
Visualization Pattern	Image	Reference (source / description)
Ghost		Lee et al. [3] (situated visualization taxonomy: target-state preview); implemented here as a reusable feedback component.
Arrow (Glyph)		Lee et al. [3] (attention guidance via glyphs/arrows); runtime off-screen pointing logic in this thesis.
Label		Lee et al. [3] (contextual overlays/panels); serialized configuration via Gamification Profiles in this thesis.
Trajectory Line		Lee et al. [3] (path/trajectory visualization); coupled to <i>Trajectory</i> interaction in this thesis.
Motion Indicator (new)		<i>This thesis</i> (new; “how to move” cues such as arcs and circular arrows).
Progress Indicator (new)		<i>This thesis</i> (new; continuous/timed progress bars linked to action state).

Table A.2: Visualization/Gamification Pattern catalog with thumbnails and provenance references.

B Gamification Elements Setup Tutorials

The following subsections describe the configuration and runtime behavior of the main gamification elements embedded in the framework. Each example demonstrates how to add, configure, and customize feedback components within the **Gamification Panel** of an Action Node.

B.1 Ghost Pattern

Purpose

The *Ghost Pattern* provides spatial alignment guidance by previewing a target position for placement or assembly tasks. It helps users understand *where* an object should go before performing the action.

Setup in Unity

1. Open the **Scenegraph Editor** and select an Action Node (e.g., Insert or Use).
2. In the **Gamification Panel**, click **Add Pattern > Ghost**.
3. Configure visibility parameters:
 - **Show on Interactable** – highlights the object to grab.
 - **Show on Target** – displays a semi-transparent projection of the correct position.
4. Optionally adjust opacity, color, and fade duration in the inspector.
5. Save the Scenegraph; configuration will be serialized into the node's *Gamification-Profile*.

Runtime Behavior

When the action begins, the Ghost appears above the interactable or target area. Once the object is correctly aligned, the Ghost fades out smoothly, confirming successful positioning.

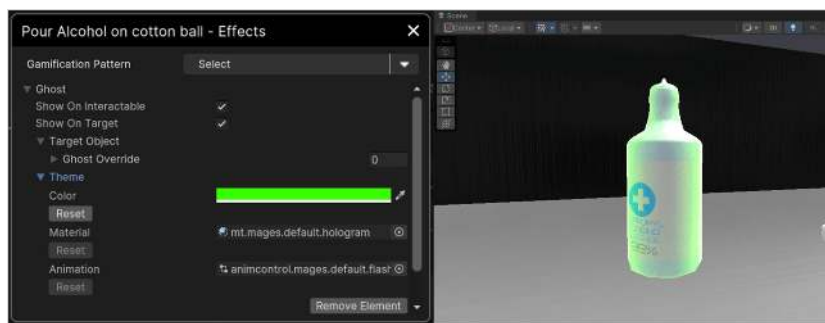


Figure A.1: Example of a Ghost pattern previewing correct placement alignment.

B.2 Arrow Pattern

Purpose

The *Arrow Pattern* directs the learner's attention to off-screen or distant targets, improving spatial orientation.

Setup in Unity

1. Select an Action Node (e.g., Use or Question).
2. Open the **Gamification Panel** and add an **Arrow Pattern**.
3. Choose **Show on Interactable** or **Show on Target**.
4. Optionally set a custom arrow prefab, color gradient, or rotation speed.

Runtime Behavior

If a target leaves the learner's field of view, the Arrow appears at the screen edge and points toward it. The cue disappears when the target re-enters view.

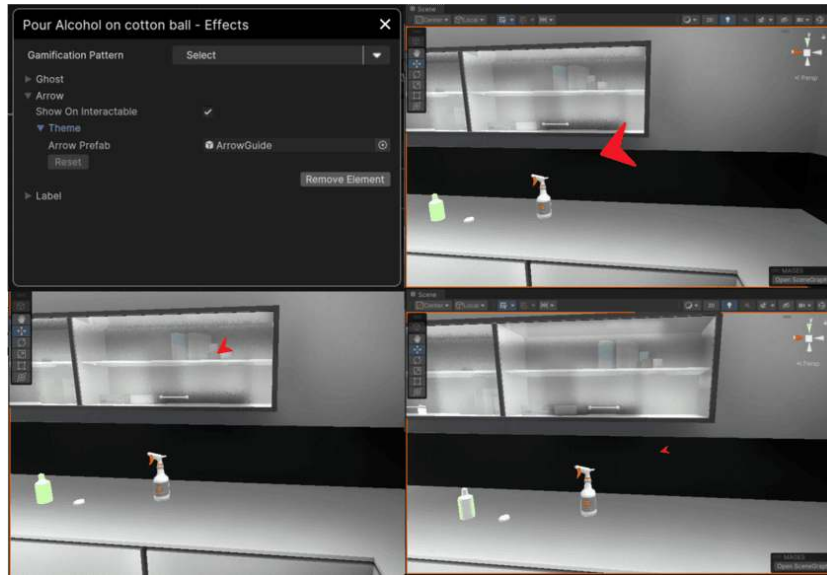


Figure A.2: Arrow pattern dynamically pointing to an off-screen object.

B.3 Label Pattern

Purpose

The *Label Pattern* provides contextual information or feedback through floating text elements (e.g., “Pick up the bottle”, “Action completed”).

Setup in Unity

1. In the Action Node’s Gamification Panel, select **Add Pattern** > **Label**.
2. Set text content and visibility (on interactable, target, or both).
3. Enable **Show on Trigger** or **Show on Success** for dynamic display timing.
4. Adjust font size, color, and anchor offset as needed.

Runtime Behavior

Labels appear at key points of the task—initial instructions, mid-progress hints, or completion messages. They automatically follow the target object and fade upon completion.

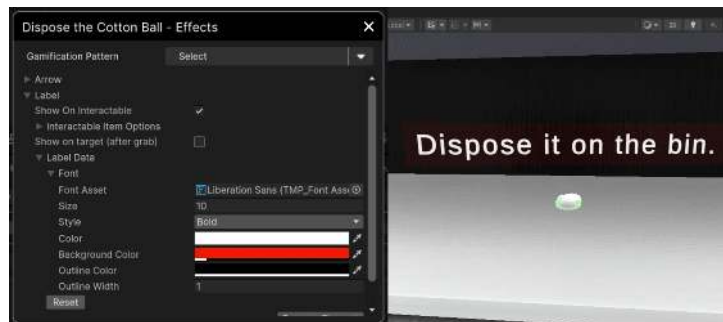


Figure A.3: Label pattern showing contextual guidance at runtime.

B.4 Progress Indicator Pattern

Purpose

The *Progress Indicator* visualizes the learner's advancement within a continuous or time-based task, reinforcing motivation and clarity.

Setup in Unity

1. Select an Action Node (e.g., Trajectory or Flow).
2. Add a **Progress Indicator Pattern** via the Gamification Panel.
3. Choose type: circular, linear, or bar.
4. Bind progress to the relevant metric (e.g., trajectory completion or activation count).

Runtime Behavior

The progress bar updates in real time (0–100%), providing quantitative feedback. It may also trigger color transitions or completion animations.

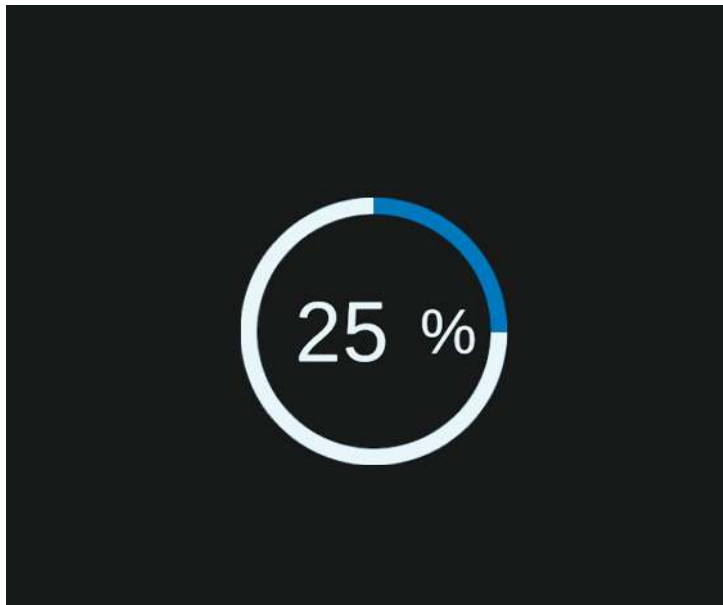


Figure A.4: Progress Indicator visualizing continuous task completion.

B.5 Motion Indicator Pattern

Purpose

The *Motion Indicator* conveys *how* a movement should be performed, using animated glyphs or arcs to illustrate direction or rotation.

Setup in Unity

1. From the Gamification Panel, select **Add Pattern > Motion Indicator**.
2. Configure motion type (linear, rotational, circular).
3. Define target direction vectors or rotation angles.
4. Adjust animation speed and looping.

Runtime Behavior

The Motion Indicator appears near the tool or target, animating the required motion path. It disappears once the action is correctly executed.



Figure A.5: Motion Indicator illustrating correct rotational gesture.

C Interaction Pattern Setup Tutorials

C.1 Activate Action

Purpose

The *Activate Action* enables users to interact with tools or instruments that perform an effect when actively engaged, such as pressing a button, pumping a syringe, or spraying disinfectant. In this example, the user must pick up a spray bottle and press its trigger three times to complete the action.

Setup in Unity

1. In the **Scenegraph Editor**, right-click and select **New Action > Activate Action**. A new Action Node appears. Connect it with the preceding and following nodes in the procedural flow.
2. Click the arrow on the top-left corner of the node to expand advanced settings (Figure A.6).
3. Create the **Activate Object** — in this case, a spray bottle model that the user will hold. Ensure that physical colliders are attached to the object via Unity's **Add Component > Collider**. Adjust collider shapes to fit the mesh accurately (Figure A.6).
4. (Optional) add an **animation** to visualize the activation event, e.g., movement of the spray lever or emission of particles. Link this animation to the object's **Animator** component.
5. Configure **hand poses**:
 - *Selection Pose* – for natural grabbing.
 - *Activation Pose* – for pressing or trigger-pulling.

These are defined in the **Grabbable** component and can smoothly transition using pose lerping.

6. Set **Number of Activations** = 3. Optionally toggle **Activate on Press** to true if activation should occur on trigger press instead of release.
7. Save the **Activate Object** as a prefab under **Assets > Resources > Prefabs**, then drag it into the node's **Activate Object** field. Name the Action, e.g., *Activate the spray bottle*.
8. Remove any test prefabs from the scene and save the Scenegraph.

Runtime Behavior

At runtime, the system spawns the spray bottle prefab. The learner must grab the bottle and press the trigger button three times. Each valid activation increments an internal counter; after the final activation, **OnSuccess()** fires, progressing the Scenegraph.

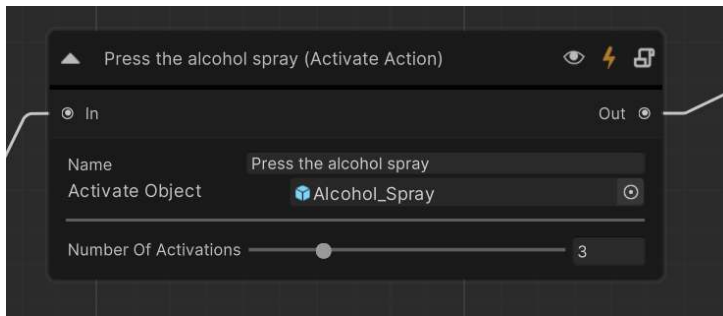


Figure A.6: Activate Action node configuration in the Scenegraph Editor.

C.2 Trajectory Action

Purpose

The *Trajectory Action* is used to simulate continuous spatial motions such as cutting, tracing, or drawing. This tutorial demonstrates how to configure a cutting motion on a fake skin model using a scalpel that follows a defined trajectory.

Setup in Unity

1. In the **Scenegraph Editor**, right-click and select **New Action > Trajectory Action**. The new node appears in the Scenegraph (Figure A.7).
2. Create the **Interactable Object** — e.g., a scalpel. Add appropriate collider components to the object, ensuring they accurately match the mesh. Save the scalpel as a prefab and drag it to the node's **Object Used** field.
3. Press **New Trajectory Line**. A spline-based template appears in front of the scene camera. Position it approximately along the surface where the cutting should occur.
4. Open Unity's **Spline Editor Tool** (top-left of Scene view). Adjust spline knots to define the desired motion path. Add new knots by selecting an existing one and pressing the + button; configure them as *Auto* for smooth interpolation.
5. In the **SplineInstantiate** component, adjust **Instantiation Count** to define the number of visible segments (e.g., 9). Morph the spline to match the desired cut shape.
6. Save the trajectory line as a prefab and assign it to the node's **Trajectory Line** field.

7. (Optional) Add an animated object (e.g., fake skin) with an animator state, such as `SkinIncision`. Link this object under the node's **Animated Object** field and select the animation state from the dropdown. The animation will synchronize with the user's motion along the spline.
8. Name the Action, e.g., *Cut the fake skin*, and save the Scenegraph.

Runtime Behavior

When executed, the scalpel spawns and the user can grab it to begin the cut. As the tool collider moves along the spline, progress is updated dynamically and drives the linked animation. The Action completes when the full trajectory is traversed, triggering the `OnSuccess()` event.

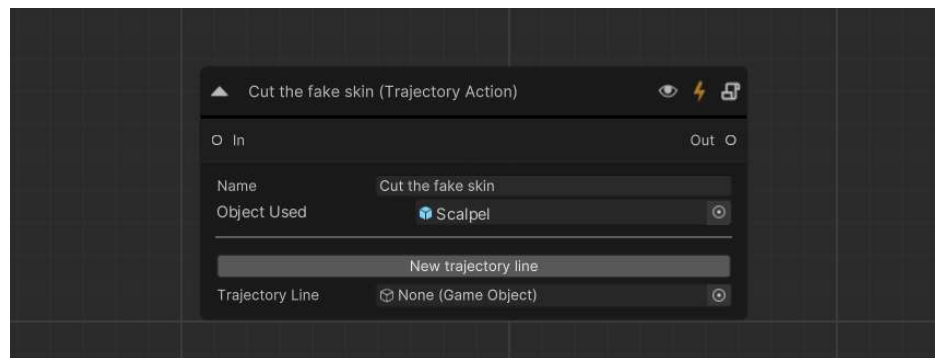


Figure A.7: Trajectory Action node configuration in the Scenegraph Editor.

C.3 Question Action

Purpose

The *Question Action* integrates in-situ quizzes or reflective checkpoints within procedural training flows. It presents a question and possible answers directly in XR, supporting single or multiple selections.

Setup in Unity

1. In the **Scenegraph Editor**, select **New Action > Question Action**.
2. Click **New Question** to spawn a question template in front of the scene camera.
3. Position the question interface in the desired spatial location.
4. Enter the question text (e.g., “In which part of the body should the injection be given?”).
5. Add answer options and mark one or more as correct.
6. Use the **Preview Answers** button to visualize layout during editing.
7. Save the question and connect the node to the next procedural step.

Runtime Behavior

When triggered, a floating UI panel appears in front of the learner. The user selects answers using ray or gaze input and submits the response. The system validates correctness, provides feedback, and triggers `OnSuccess()` upon completion. To modify questions later, reopen the node and click **Edit Question**.

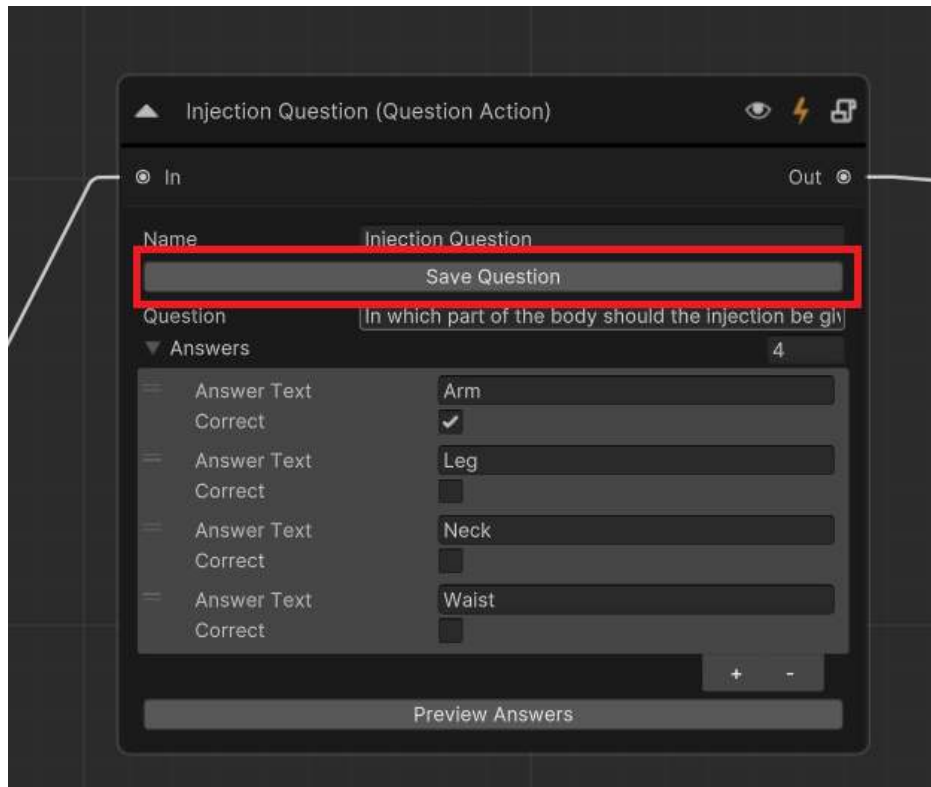


Figure A.8: Question Action node configuration in the Scenegraph Editor.

C.4 Choice Action

Purpose

The *Choice Action* introduces branching logic, allowing learners to select between multiple paths or actions (e.g., choosing whether to sterilize an incision or place a tool on a tray).

Setup in Unity

1. In the **Scenegraph Editor**, select **New Action > Choice Action**. The node appears with multiple output paths (Figure A.9).
2. Add or remove available paths with the + and - buttons.
3. Create a **Choice Object**, typically a UI panel with buttons. Attach a script implementing the `ChoiceActionData.IDecisionHandler` interface:

```
public class ChoiceActionUI : MonoBehaviour, ChoiceActionData.IDecisionHandler {
    private Action<int> callbackAction;
    public void SetCallbackHandle(Action<int> completeAction) {
        callbackAction = completeAction;
    }
    public void SelectChoice(int choice) {
        callbackAction.Invoke(choice);
    }
}
```

4. Assign the prefab to the node's **Choice Object** field. Configure each button to call `SelectChoice()` with the corresponding path index (0, 1, ...).
5. Save and name the Action, e.g., *Choose next step*, then connect the output paths to subsequent Actions.

Runtime Behavior

At runtime, the UI spawns with selectable options. Choosing an option invokes the associated path in the Scenegraph—e.g., “Use” or “Insert” Actions. This mechanism enables adaptive, decision-driven learning scenarios.

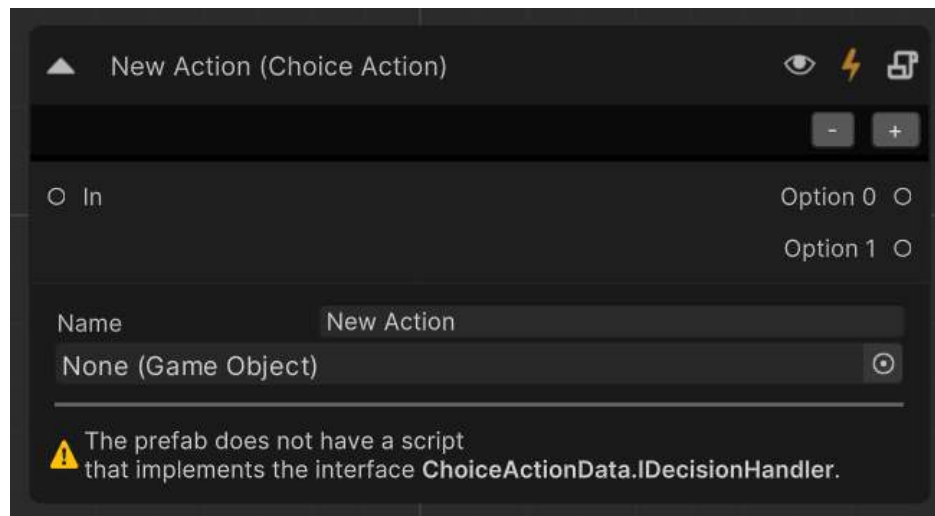


Figure A.9: Choice Action node configuration in the Scenegraph Editor.

D Algorithmic Heuristics and Runtime Manifest

The *algorithmic heuristics* embedded in the Gamification Subsystem formalize how feedback and guidance events are triggered, updated, and dismissed during runtime. Each heuristic represents a rule that evaluates contextual conditions—such as object alignment, visibility, or user action state—and activates corresponding feedback behaviors declaratively.

D.1 Definition

An **algorithmic heuristic** is a parameterized rule of the form:

```
if <condition> then <action> [else <alternative>]
```

where the *condition* refers to system state (e.g., object visibility, alignment tolerance, success criteria) and the *action* defines feedback operations (e.g., show, hide, fade, color change). These rules are evaluated dynamically by the runtime engine, removing the need for hard-coded scripting.

Example: Heuristic Set for an Insert Action

The following manifest excerpt shows the declarative representation of a typical Insert Action combining Ghost, Arrow, and Label feedback components:

```
"gamificationProfile": {
  "patterns": ["Ghost", "Arrow", "Label"],
  "rules": [
    {"event": "OnStart", "actions": ["Ghost.Show(target)"]},
    {"event": "OnUpdate", "cond": "!InFOV(target)",
     "actions": ["Arrow.Show(target)", "else": ["Arrow.Hide()"]},
    {"event": "OnAlign", "actions": ["Ghost.Hide()"]},
    {"event": "OnSuccess", "actions": ["Label.Show('Done')"]}
  ]
}
```

Each rule corresponds to a specific runtime event:

- **OnStart** – Initializes the feedback sequence.
- **OnUpdate** – Continuously checks the learner’s field of view and adjusts the Arrow.
- **OnAlign** – Detects correct object alignment and hides the Ghost.
- **OnSuccess** – Displays completion feedback via the Label.

Generalization Across Actions

While this example focuses on the *Insert* Action, identical heuristic logic applies to all other interaction types (*Use*, *Remove*, *Activate*, *Trajectory*, *Question*, and *Choice*). Each Action Node references its own **GamificationProfile**, containing pattern definitions and rules serialized in JSON format. At runtime, these profiles are parsed by the pattern manager, ensuring consistent, context-aware behavior across scenarios.

Runtime Integration

The heuristic engine operates as a middleware layer between the Scenegraph and the instantiated pattern prefabs. It listens to Action Node events (**OnStart()**, **OnUpdate()**, **OnAlign()**, **OnSuccess()**) and executes the declared rule set automatically, maintaining deterministic, reproducible behavior across all gamified training scenarios.